AWS Infrastructure Documentation — DevNotes

1. IAM (Identity and Access Management)

Purpose: IAM helps manage access to AWS resources by controlling identities, permissions, and authentication.

Screenshot 1 — Root User Security Credentials

What I did: Reviewed the AWS root user's security credentials, MFA configuration, and access-key status.

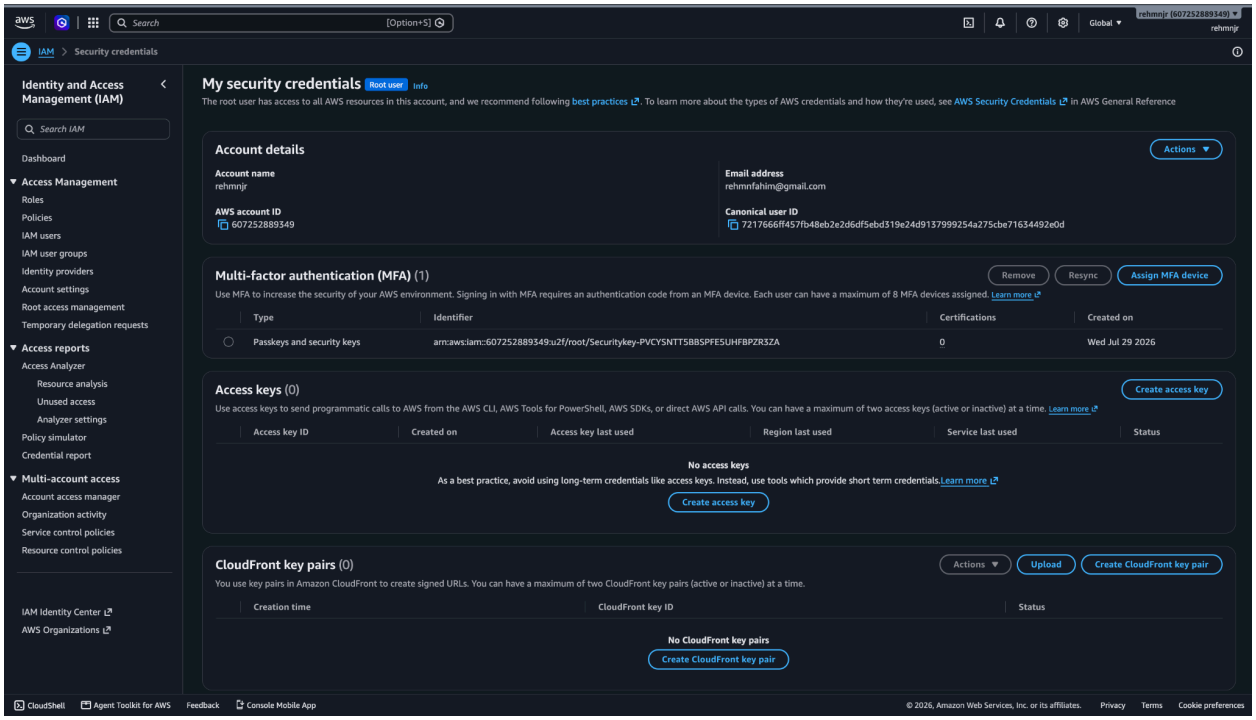
Why did it: To verify account-level authentication security and check whether long-term root access keys were present.

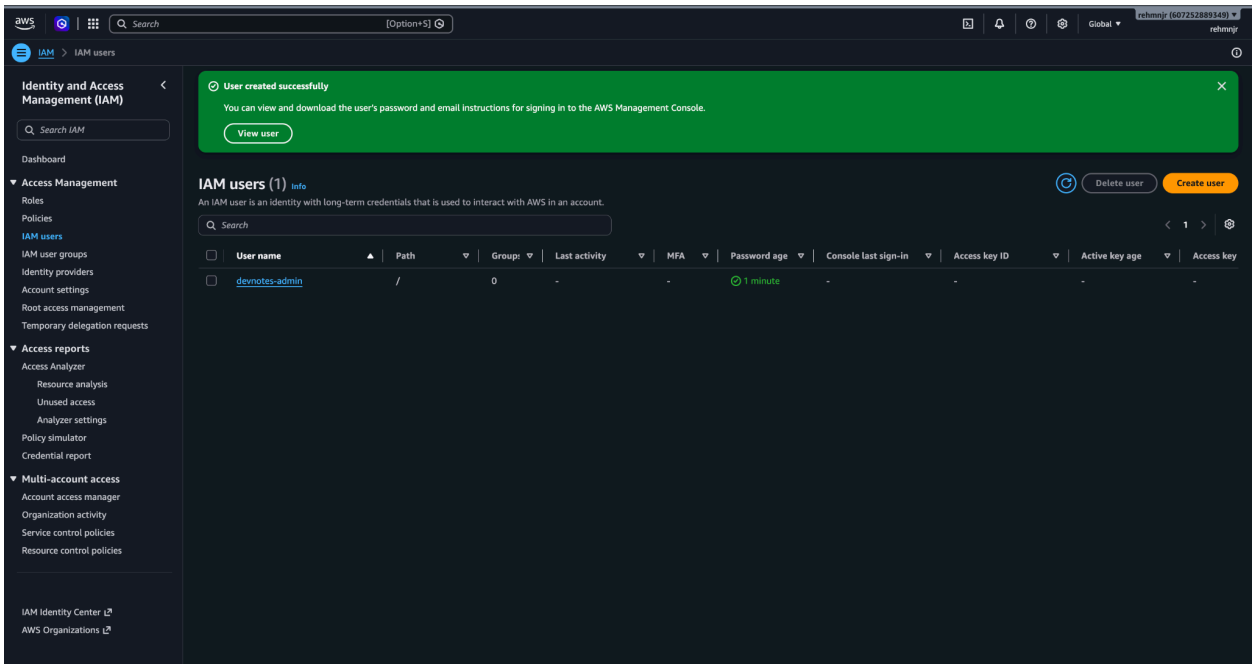
Configuration:

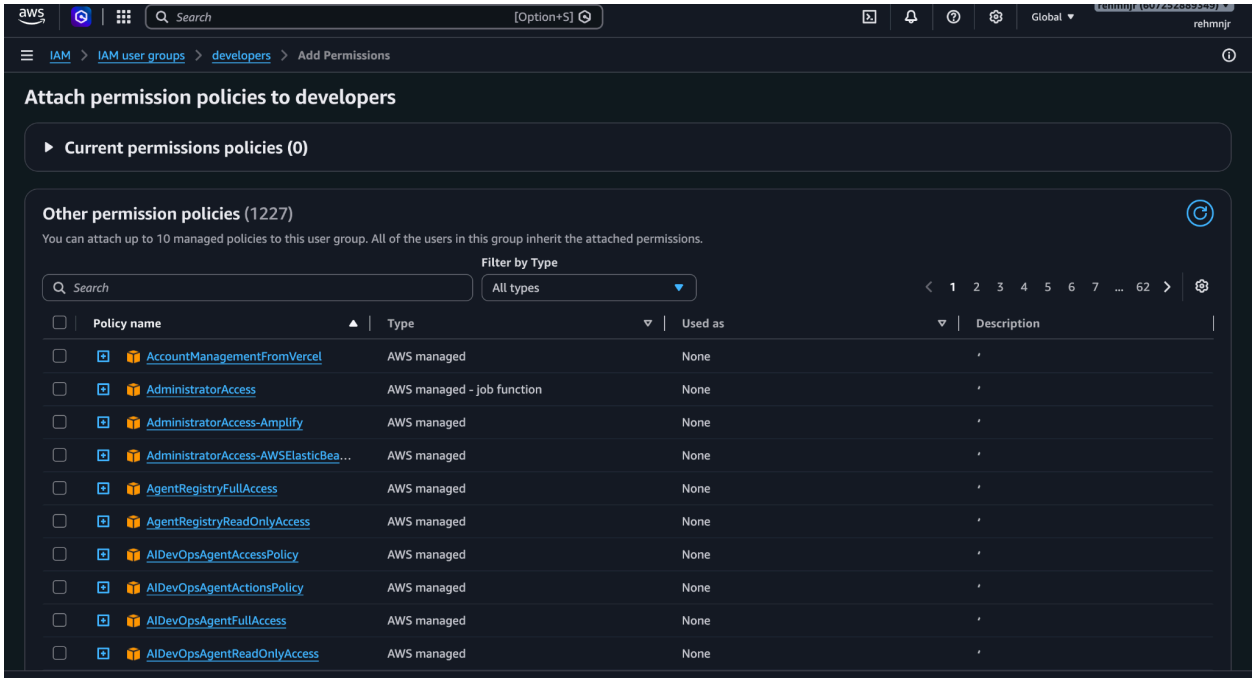
- Multi-factor authentication (MFA): A passkey/security key is listed.
- Root user access keys: None listed.
- CloudFront key pairs: None listed.

Result: MFA is configured for the root user, and no root access keys are listed in this screenshot.

Security note: The root user should be reserved for tasks that specifically require root access. Use an IAM identity with appropriate permissions for routine administration. Never publish account identifiers or sensitive credential details unnecessarily.


Search
[Option+S]
Global
rehmnjr (607252889349)
rehmnjr
IAM > Security credentials
Identity and Access Management (IAM)
Search IAM
Dashboard
Access Management
Roles
Policies
IAM users
IAM user groups
Identity providers
Account settings
Root access management
Temporary delegation requests
Access reports
Access Analyzer
Resource analysis
Unused access
Analyzer settings
Policy simulator
Credential report
Multi-account access
Account access manager
Organization activity
Service control policies
Resource control policies
IAM Identity Center
AWS Organizations
My security credentials
Root user
Info
The root user has access to all AWS resources in this account, and we recommend following best practices. To learn more about the types of AWS credentials and how they're used, see AWS Security Credentials in AWS General Reference
Account details
Actions
Account name
rehmnjr
Email address
rehmnfahim@gmail.com
AWS account ID
607252889349
Canonical user ID
7217666ff457fb48eb2e2d6df5ebd319e24d9137999254a275cbe71634492e0d
Multi-factor authentication (MFA) (1)
Remove
Resync
Assign MFA device
Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 8 MFA devices assigned. Learn more
Type
Identifier
Certifications
Created on
Passkeys and security keys
arn:aws:iam::607252889349:u2f/root/Securitykey-PVCYSNTT5BBSPFE5UHFBPZR3ZA
0
Wed Jul 29 2026
Access keys (0)
Create access key
Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. Learn more
Access key ID
Created on
Access key last used
Region last used
Service last used
Status
No access keys
As a best practice, avoid using long-term credentials like access keys. Instead, use tools which provide short term credentials. Learn more
Create access key
CloudFront key pairs (0)
Actions
Upload
Create CloudFront key pair
You use key pairs in Amazon CloudFront to create signed URLs. You can have a maximum of two CloudFront key pairs (active or inactive) at a time.
Creation time
CloudFront key ID
Status
No CloudFront key pairs
Create CloudFront key pair
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Search
[Option+S]
Global
rehmnjr (607252889349)
rehmnjr
IAM > IAM users
Identity and Access Management (IAM)
Search IAM
Dashboard
Access Management
Roles
Policies
IAM users
IAM user groups
Identity providers
Account settings
Root access management
Temporary delegation requests
Access reports
Access Analyzer
Resource analysis
Unused access
Analyzer settings
Policy simulator
Credential report
Multi-account access
Account access manager
Organization activity
Service control policies
Resource control policies
IAM Identity Center
AWS Organizations
User created successfully
You can view and download the user's password and email instructions for signing in to the AWS Management Console.
View user
IAM users (1) Info
An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.
Delete user
Create user
Search
User name
Path
Group
Last activity
MFA
Password age
Console last sign-in
Access key ID
Active key age
Access key
devnotes-admin
/
0
-
-
1 minute
-
-
-
-



IAM User Group and Permissions

What I did: Created an IAM user group named `developers` and attached the AWS-managed `AmazonEC2ReadOnlyAccess` policy.

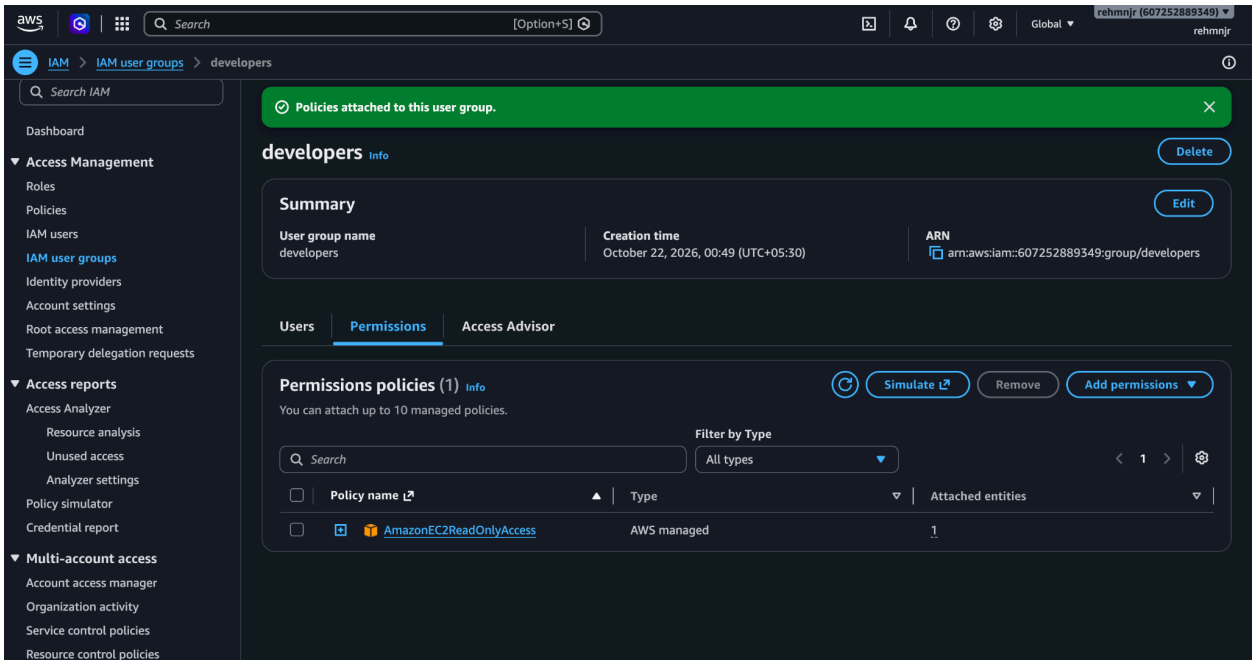
Why did it: To organize developer identities and manage permissions at the group level instead of assigning the same permissions individually to every user.

Configuration:

- **Group name:** `developers`
- **Attached policy:** `AmazonEC2ReadOnlyAccess`
- **Policy type:** AWS managed
- **Purpose:** Read-only access to supported EC2 resources.

Result: The policy was successfully attached to the group.

Security note: This policy does not grant permission to create, modify, or terminate EC2 instances. Group membership and the permissions of individual users should be reviewed separately.



Screenshot 2 — EC2 IAM Role

What I did: Created an IAM role named `DevNotes-EC2-S3-Role` trusted by the Amazon EC2 service.

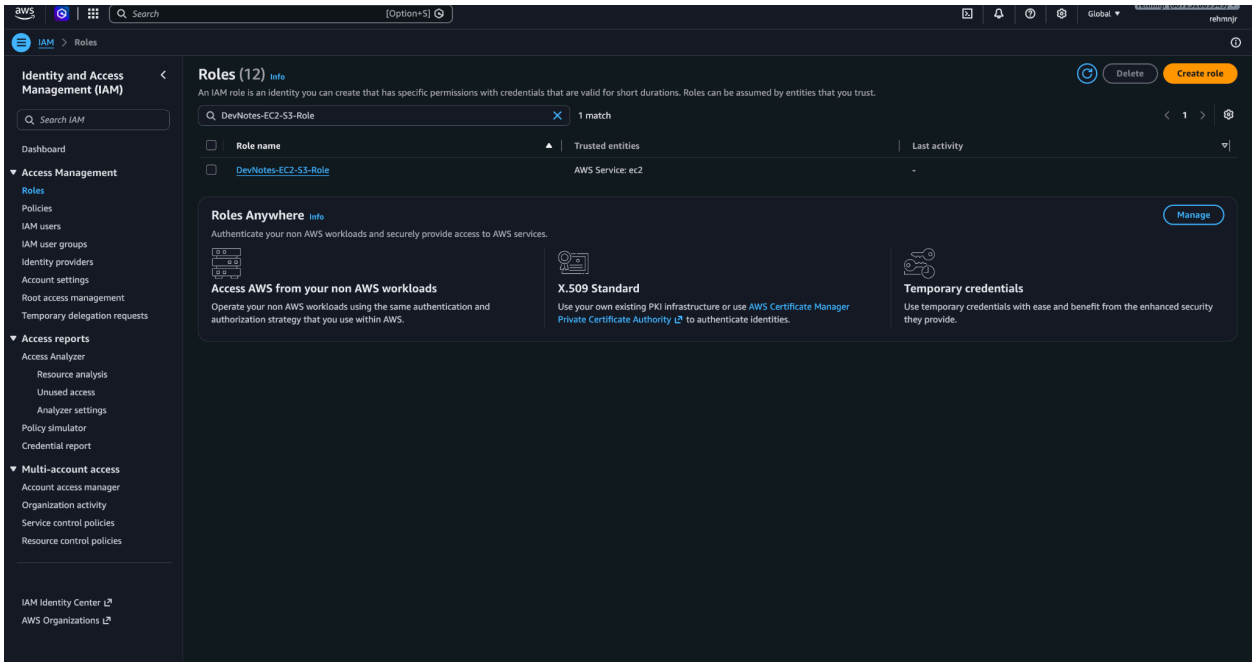
Why did it: To allow the EC2 instance to access permitted AWS services without storing long-term AWS access keys on the server.

Configuration:

- **Role name:** `DevNotes-EC2-S3-Role`
- **Trusted entity:** AWS Service — EC2
- **Intended use:** Grant the DevNotes EC2 instance permissions to access its required AWS resources, including S3 backups.

Result: The IAM role is present in AWS and configured to trust the EC2 service.

Note: This screenshot confirms the role and its trusted service, but does not show the attached policies. The actual S3 permissions must be verified in the role's **Permissions** tab.



---

2. AWS Budgets — Cost Monitoring

Screenshot 3 — Monthly Cost Budget Configuration

What I did: Configured a monthly cost budget named `DevNotes-Monthly-Budget`.

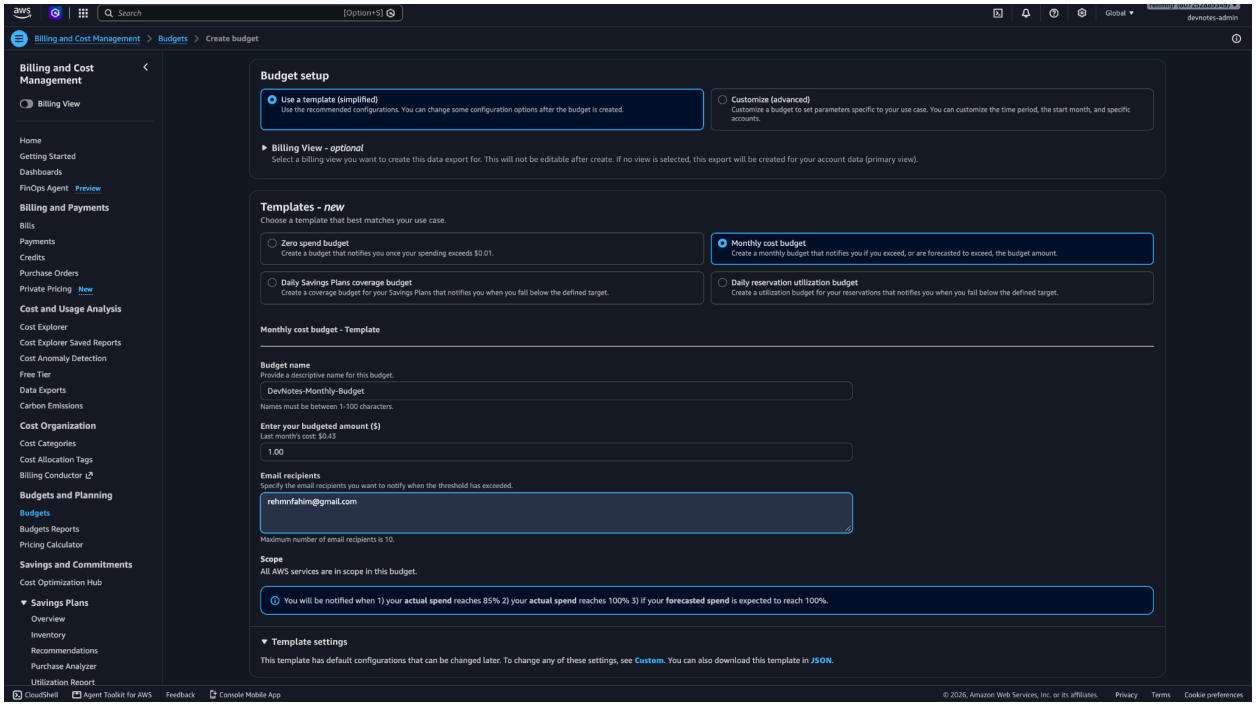
Why did it: To receive email notifications when AWS spending reaches or is forecast to exceed the configured budget.

Configuration:

- **Budget name:** `DevNotes-Monthly-Budget`
- **Budget type:** Monthly cost budget
- **Budget amount:** $1.00
- **Scope:** All AWS services
- **Notification recipient:** Configured email address
- **Alert thresholds:** 85% actual spending, 100% actual spending, and forecasted spending reaching 100%.

Result: The budget configuration is designed to alert the account owner about project costs.

Important: This screenshot shows the budget setup form, not proof that the budget was successfully created or that notifications were delivered. AWS Budgets sends alerts; it does not automatically stop resources when the threshold is exceeded.



---

3. Amazon VPC — Network Infrastructure

Screenshot 4 — DevNotes VPC

What I did: Created a dedicated Virtual Private Cloud (VPC) named `DevNotes-VPC-vpc`.

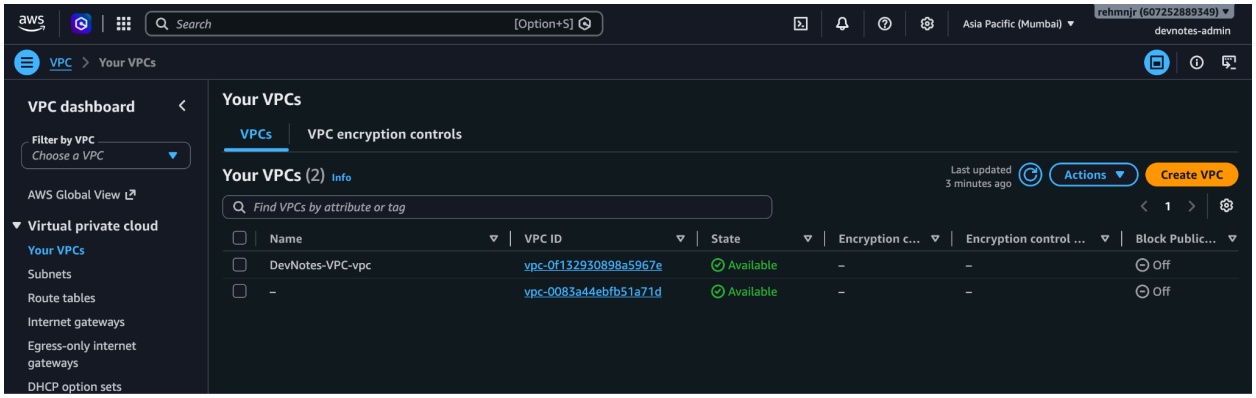
Why did it: To isolate the application's AWS networking resources and control communication between the web server and database.

Configuration:

- **VPC name:** `DevNotes-VPC-vpc`
- **VPC ID:** `vpc-0f132930898a5967e`
- **Status:** Available
- **Region:** Asia Pacific (Mumbai), `ap-south-1`

Result: The dedicated VPC is available for the DevNotes infrastructure.

Note: The screenshot shows two VPCs in the account. Only the named DevNotes VPC is identified here as part of this project; the other VPC should not be assumed to be project-related.



Screenshot 5 — VPC Subnets Configuration

What I did: Configured subnets within the `DevNotes-VPC-vpc` VPC, including two private subnets and a dedicated public subnet.

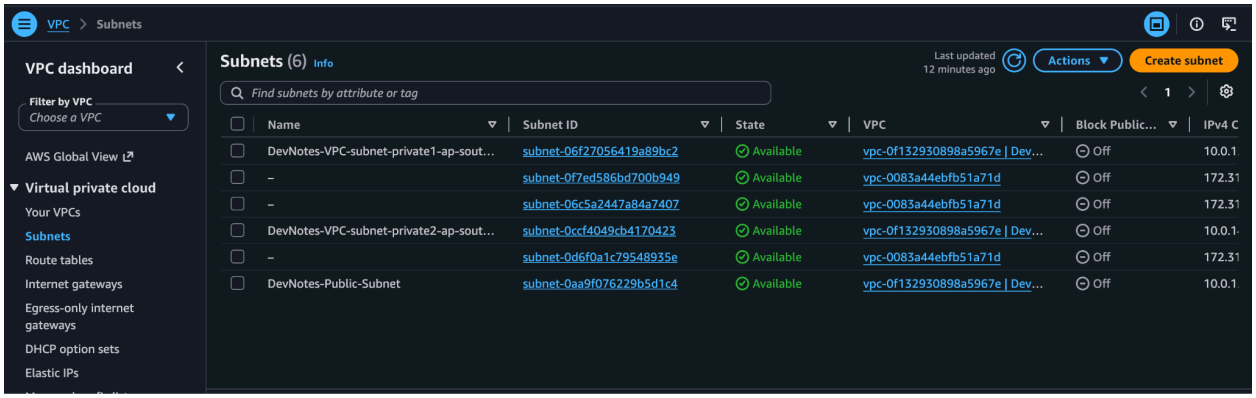
Why did it: To organize AWS resources into separate network segments and support controlled communication between the application server and database.

Configuration:

- **VPC:** `DevNotes-VPC-vpc`
- **Private subnets:** Two project subnets are listed.
- **Public subnet:** `DevNotes-Public-Subnet`
- **Status:** All listed subnets show `Available`.
- **Additional subnets:** The screenshot also shows subnets belonging to another VPC.

Result: The DevNotes VPC has separate public and private subnet resources.

Note: The screenshot truncates the CIDR ranges and does not show route-table associations. Public accessibility must be verified using the subnet's route table and public IP configuration, not just its name.



Screenshot 6 — VPC Route Tables

What I did: Configured separate route tables for the DevNotes VPC, including route tables associated with the two private subnets.

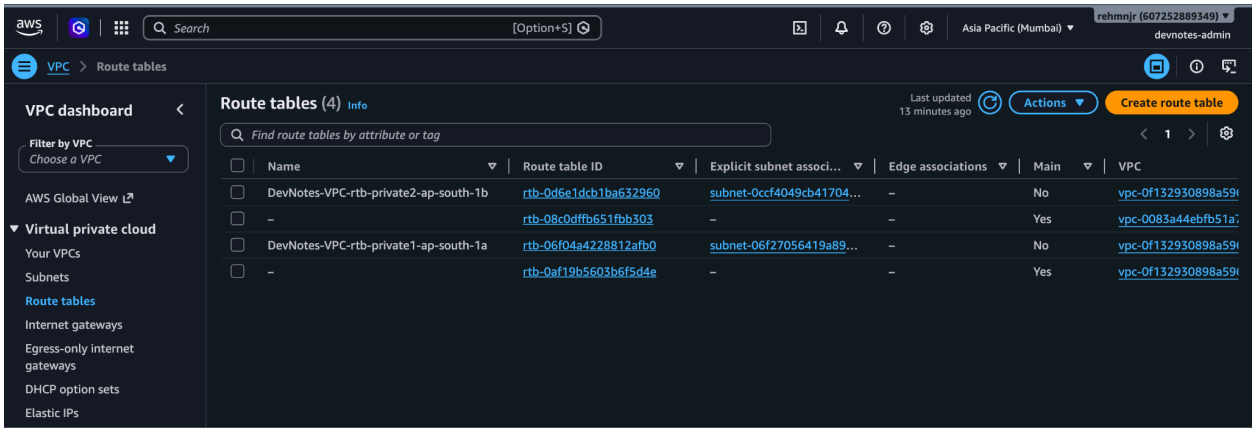
Why did it: Route tables determine where network traffic is directed, allowing public and private subnets to follow different routing rules.

Configuration:

- **Private route table 1:** `DevNotes-VPC-rtb-private1-ap-south-1a`
- **Private route table 2:** `DevNotes-VPC-rtb-private2-ap-south-1b`
- **Subnet associations:** The two named private route tables show explicit subnet associations.
- **Additional route tables:** The screenshot also shows main route tables for the DevNotes VPC and another VPC.

Result: Separate route tables are available for the DevNotes private subnets.

Note: This list view does not show the destination routes. The route table's **Routes** tab must be checked to confirm whether a default route points to an Internet Gateway, NAT Gateway, or another target.



Screenshot 7 — Internet Gateway Configuration

What I did: Created an Internet Gateway named `DevNotes-IGW` and attached it to `DevNotes-VPC-vpc`.

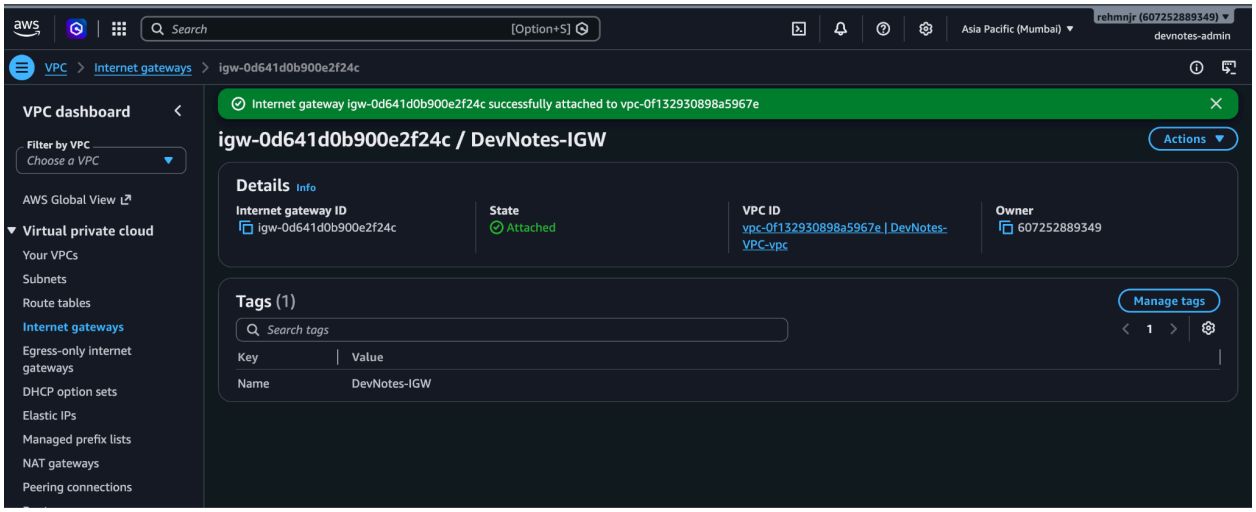
Why did it: An Internet Gateway enables internet connectivity for resources in the VPC when the appropriate routes, public IP addressing, and security rules are configured.

Configuration:

- **Name:** `DevNotes-IGW`
- **Gateway ID:** `igw-0d641d0b900e2f24c`
- **Attached VPC:** `DevNotes-VPC-vpc`
- **Status:** Attached

Result: The Internet Gateway is successfully attached to the DevNotes VPC.

Important: Attaching an Internet Gateway alone does not make a subnet public. The route table must include a route such as `0.0.0.0/0` pointing to this gateway, and the instance must have suitable public addressing and security-group rules.



Screenshot 8 — EC2 Web Server Security Group

What I did: Created a security group named `DevNotes-Web-SG` for the DevNotes web server.

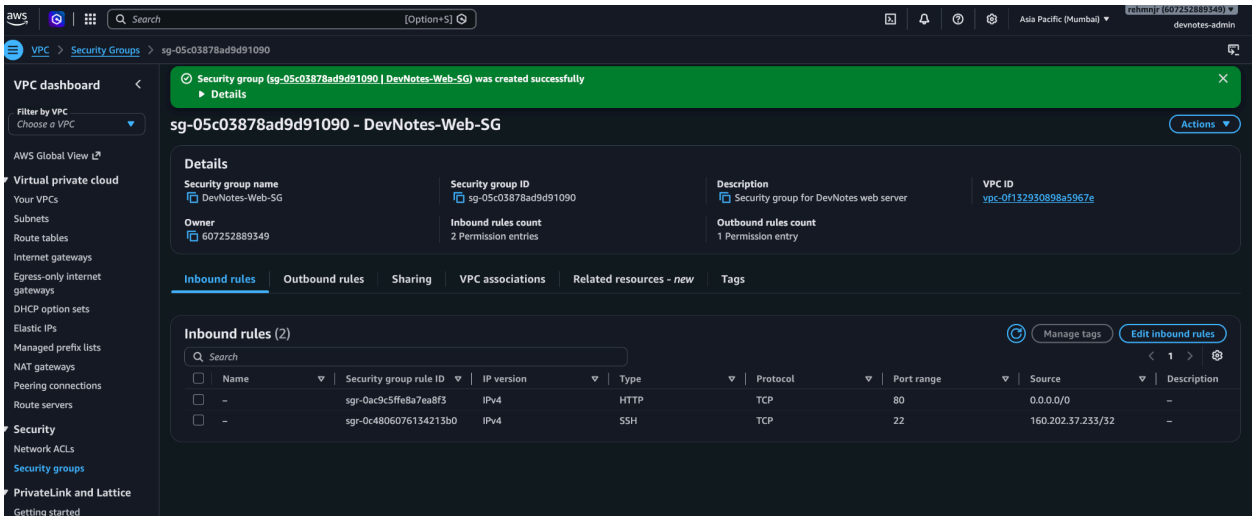
Why did it: To control incoming network traffic to the EC2 instance.

Configuration:

Protocol	Port	Source	Purpose
HTTP	80	`0.0.0.0/0`	Allows public HTTP access to the web application.
SSH	22	`160.202.37.233/32`	Restricts SSH access to one specific public IPv4 address.

Result: The security group was created successfully with two inbound rules.

Security note: HTTP on port 80 is publicly accessible. SSH is restricted to a single IP, which is safer than allowing SSH from `0.0.0.0/0`. Confirm that the SSH source IP remains current and trusted.



---

4. Amazon EC2 — Application Server

Screenshot 9 — DevNotes EC2 Instance Running

What I did: Launched an EC2 instance named `DevNotes-Web-Server` to host the containerized application.

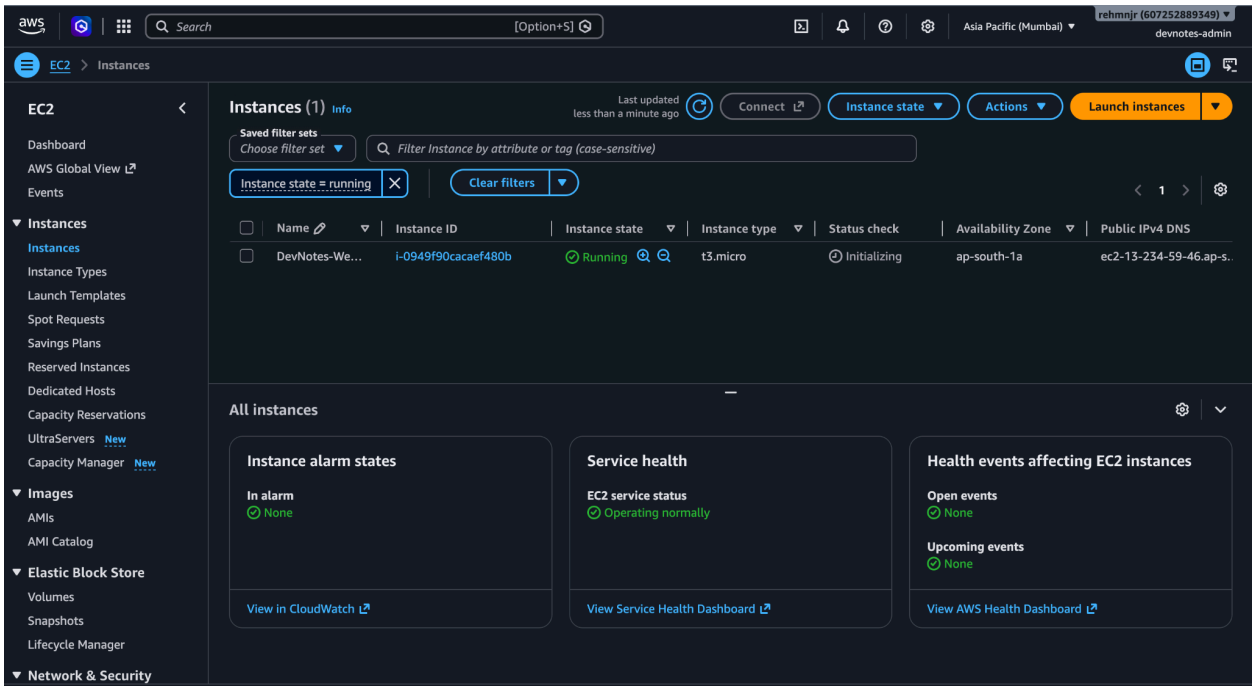
Why did it: To run the DevNotes frontend and backend containers on a cloud-based virtual server.

Configuration:

- **Instance ID:** `i-0949f90cacaef480b`
- **Instance type:** `t3.micro`
- **Availability Zone:** `ap-south-1a`
- **Instance state:** Running
- **AWS Region:** Mumbai (`ap-south-1`)

Result: The EC2 instance was running when this screenshot was captured.

Note: The instance status check was still initializing in this screenshot. A running instance does not by itself prove that the application is healthy; application and API health checks must also pass.



5. Amazon S3 — Backup Storage

Screenshot 10 — Create S3 Bucket

What I did: Created an S3 bucket named `devnotes-attachments-607252889349`.

Why did it: To store DevNotes database backup files separately from the EC2 application server, helping preserve backup copies if the server is replaced or terminated.

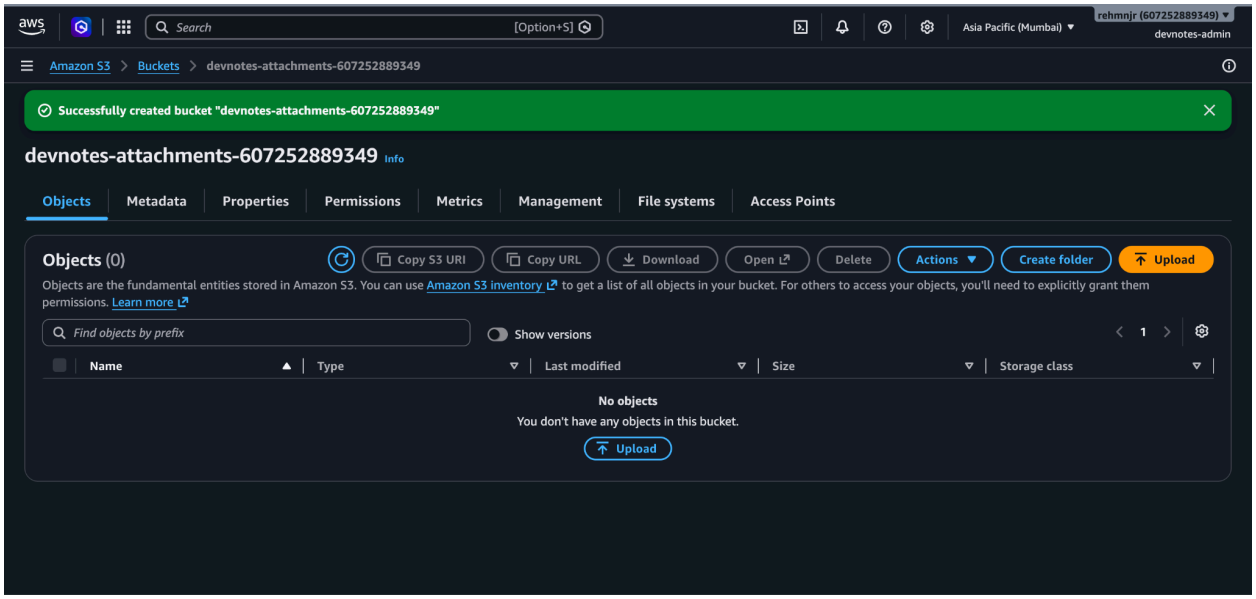
Configuration:

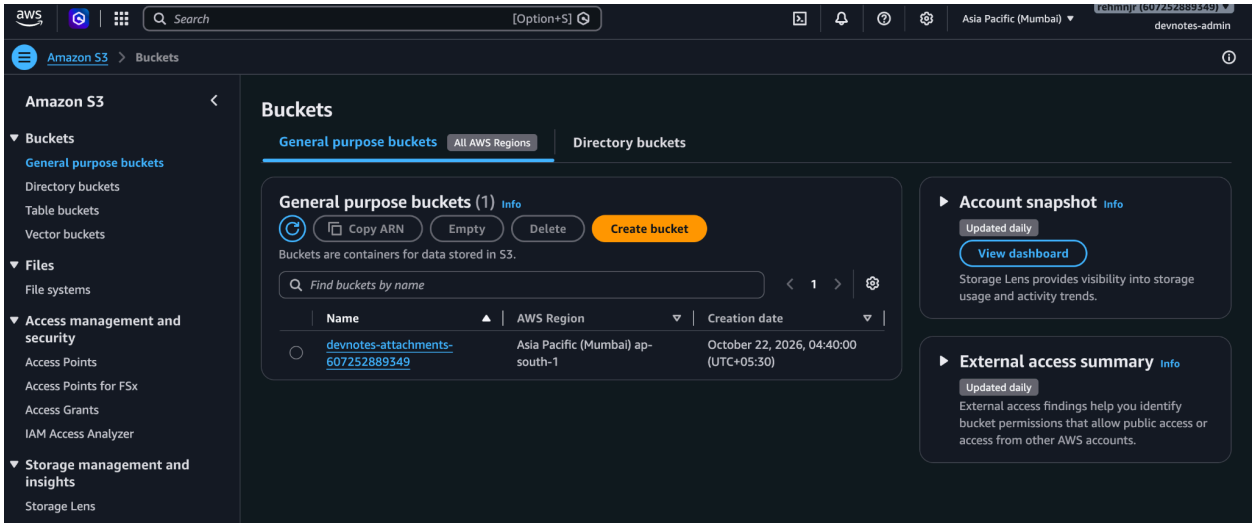
- **Bucket name:** `devnotes-attachments-607252889349`
- **AWS Region:** Asia Pacific (Mumbai), `ap-south-1`
- **Bucket creation:** Successful
- **Objects shown:** devnotes-attachments-607252889349

Result: The S3 bucket was successfully created and was initially empty.

Next configuration: The bucket was later used to store database backup files under the `backups/` prefix. Two backup files, each approximately 12 KB, were subsequently visible in that folder.

Note: This screenshot does not show the bucket's versioning, encryption, or public-access-block settings. Those should be documented from the bucket's Properties and Permissions tabs if screenshots are available.





Screenshot 11 — SSH Connection and Nginx Installation

What I did: Connected to the EC2 instance using SSH and the `DevNotes-Key.pem` key pair. Updated the Amazon Linux 2023 package metadata and installed Nginx.

Why did it: SSH provided administrative access to the server, while Nginx was installed to serve HTTP traffic and support reverse-proxy routing.

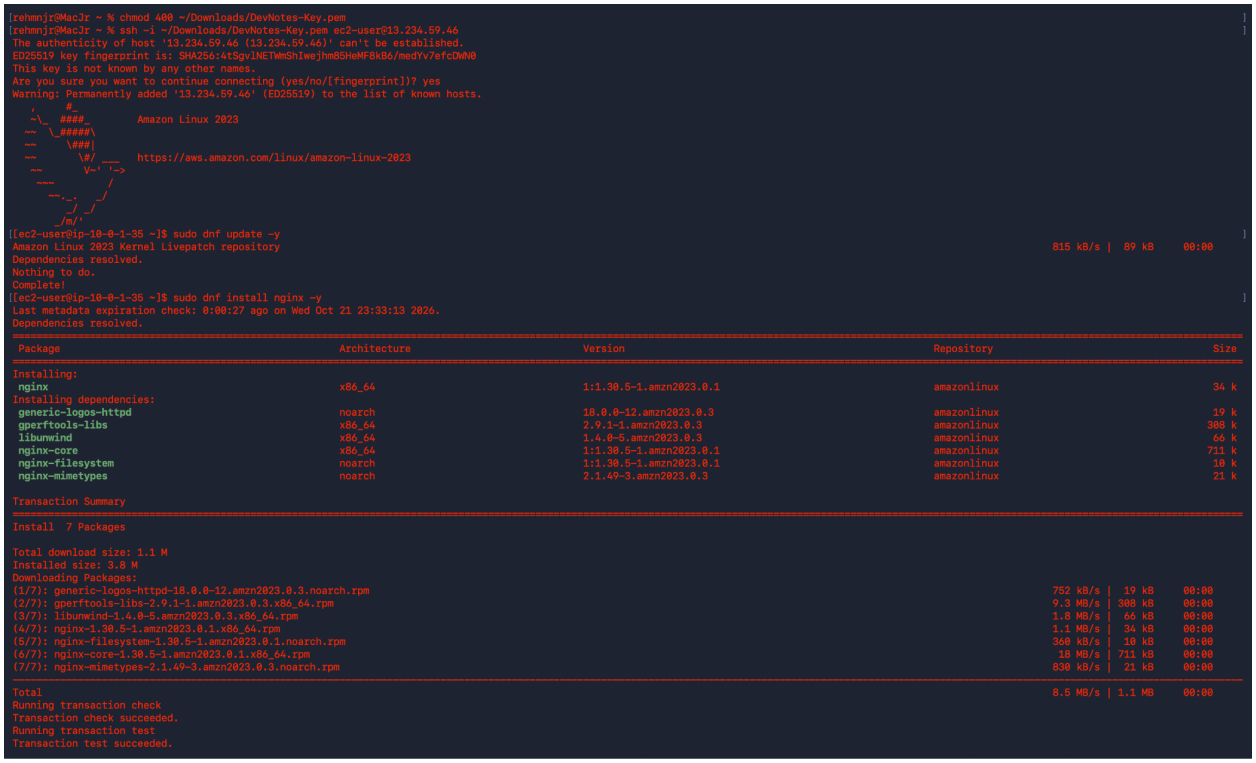
Configuration:

- **Operating system:** Amazon Linux 2023
- **SSH user:** `ec2-user`
- **SSH key:** `DevNotes-Key.pem`
- **Package manager:** `dnf`
- **Web server:** Nginx

Commands used:

chmod 400 ~/Downloads/DevNotes-Key.pem
ssh -i ~/Downloads/DevNotes-Key.pem ec2-user@13.234.59.46
sudo dnf update -y
sudo dnf install nginx -y

Result: The SSH session was established, and Nginx was installed successfully.



Screenshot 12 — Nginx Service Verification

What I did: Enabled Nginx to start automatically at boot and started the service immediately.

Why did it: To ensure that the web server runs as a managed system service and is available after a server restart.

Command used:

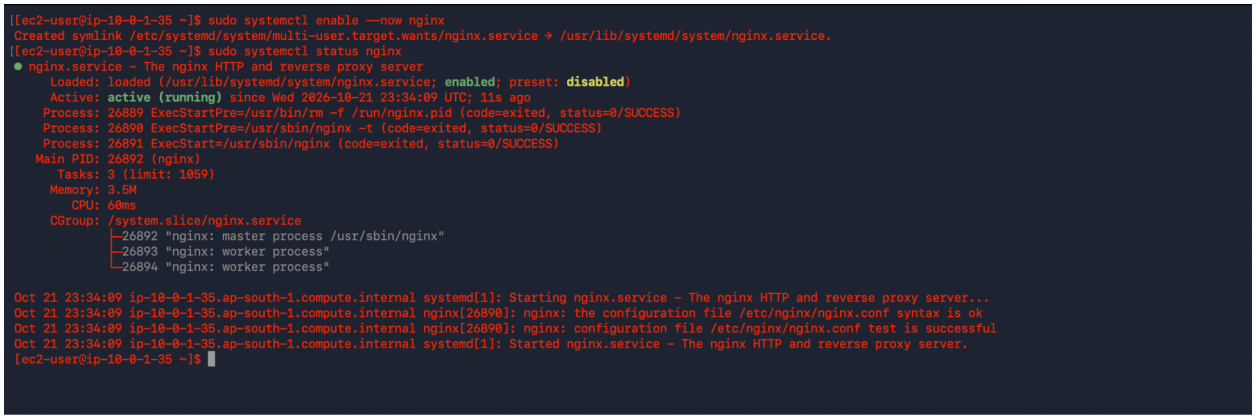
sudo systemctl enable --now nginx
sudo systemctl status nginx

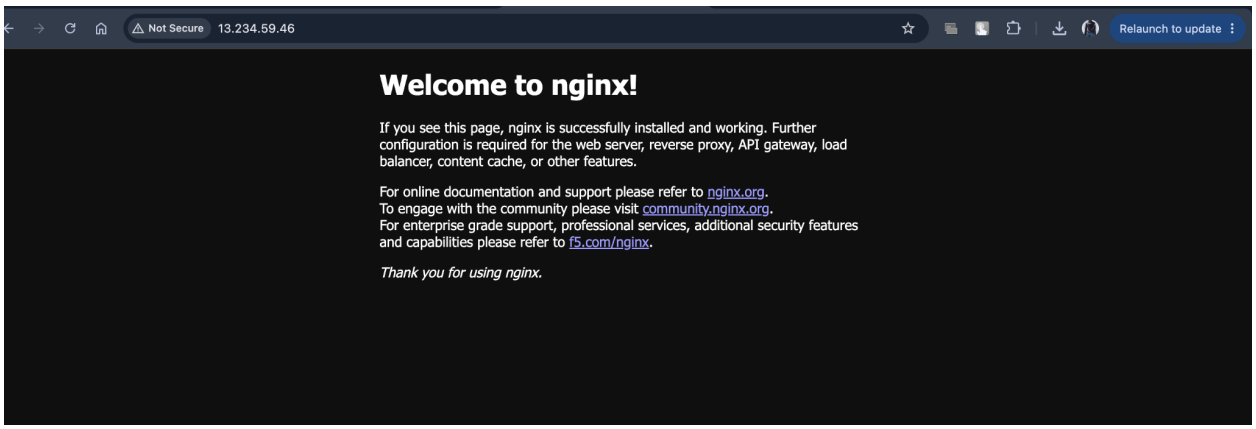
Configuration:

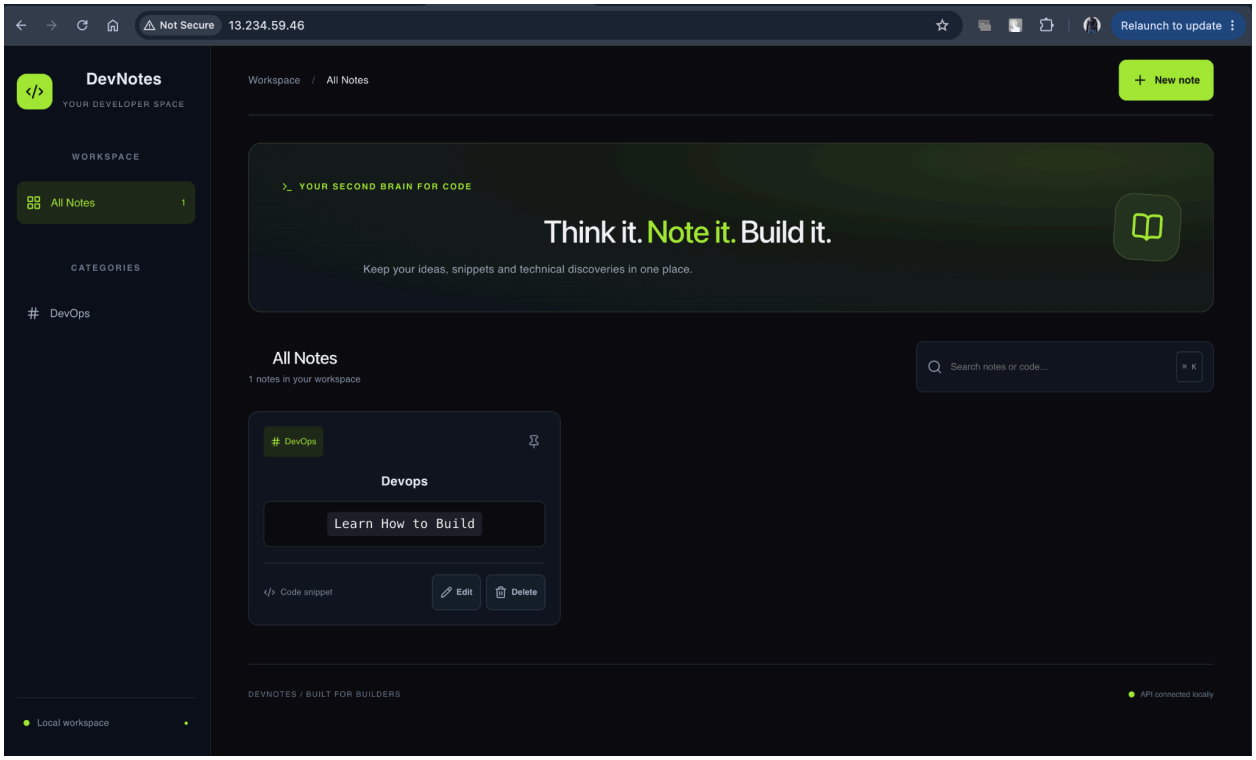
- **Service:** `nginx.service`
- **Enabled at boot:** Yes
- **Current status:** Active (running)
- **Configuration test:** Successful

Result: The screenshot confirms that Nginx was running and its configuration syntax check succeeded.

Note: This verifies the Nginx service at that point in time; it does not by itself prove that the complete DevNotes application or backend API was working.







6. Amazon CloudWatch — Monitoring and Alerts

Screenshot 13 — EC2 CPU Utilization Alarm

What I did: Created a CloudWatch alarm named `DevNotes-High-CPU-Alarm` to monitor the EC2 instance's CPU utilization.

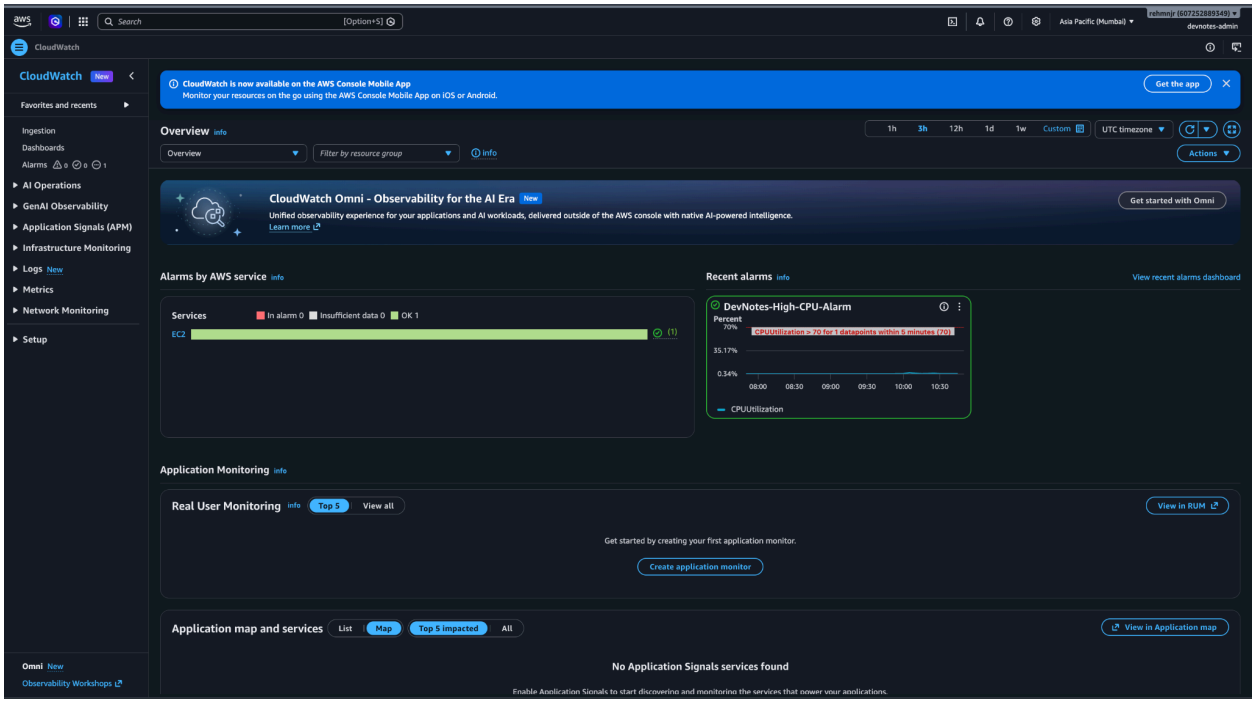
Why did it: To identify unusually high CPU usage and receive alerts when the server exceeds the configured threshold.

Configuration:

- **Alarm name:** `DevNotes-High-CPU-Alarm`
- **Monitored metric:** `CPUUtilization`
- **Threshold:** Greater than 70%
- **Evaluation period:** 1 datapoint within 5 minutes
- **Current state shown:** OK
- **Insufficient data alarms:** 0

Result: CloudWatch displays the alarm in the OK state, meaning the configured alarm condition was not currently breached based on the available evaluation data.

Note: This screenshot confirms the alarm and its threshold. It does not independently confirm that an email notification was delivered or that a high-CPU test successfully triggered the alarm.



Amazon SNS — CloudWatch Alert Notifications

Amazon Simple Notification Service (SNS) was configured to provide email notifications for the DevNotes project's CloudWatch monitoring alerts.

Screenshot 14 — SNS Topic and Email Subscription

What I did: Created the Standard SNS topic `fahim-project2-alerts` and configured an email subscription.

Why did it: To provide a notification destination for CloudWatch alarms when their configured actions are triggered.

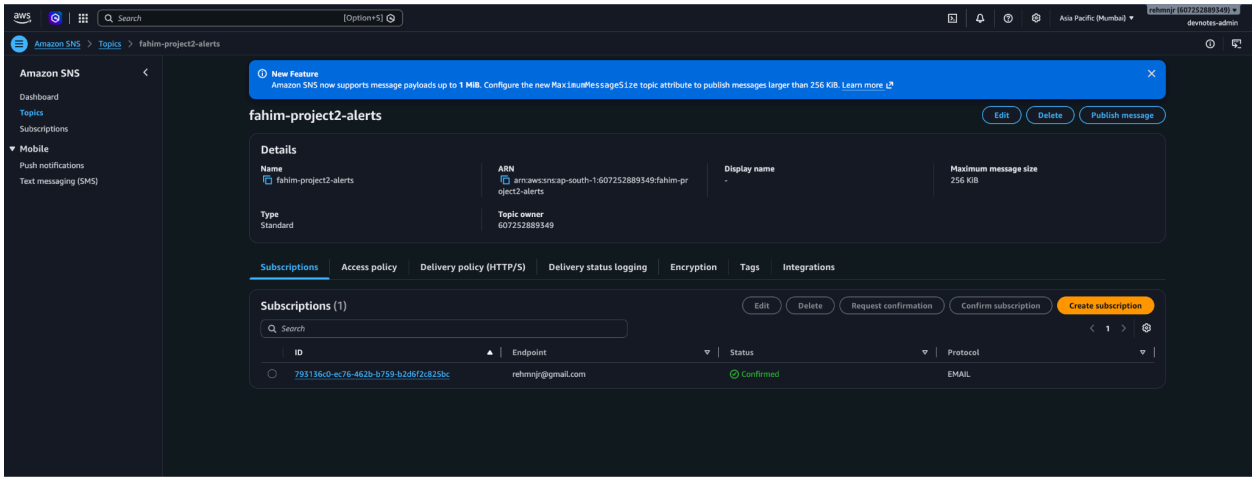
Configuration:

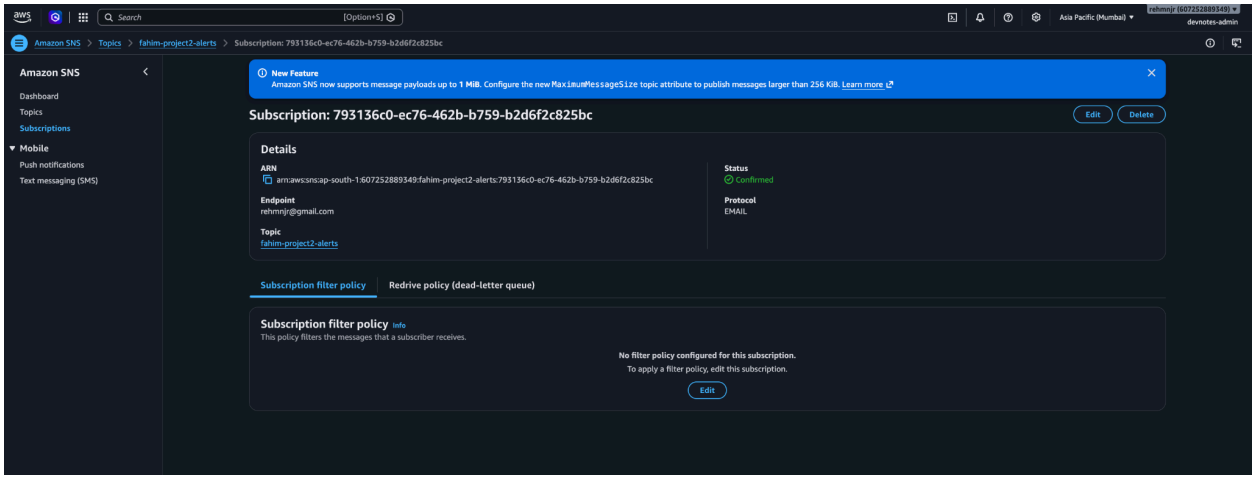
- **Topic name:** `fahim-project2-alerts`

- **Topic type:** Standard
- **Region:** Mumbai (`ap-south-1`)
- **Email endpoint:** `rehmnjr@gmail.com`
- **Subscriptions:** 1
- **Subscription status:** Confirmed

Result: The SNS topic and its email subscription were configured successfully.

Note: The screenshots confirm the topic and subscription status. They do not independently prove that an alarm notification was delivered to the inbox.





Amazon CloudWatch — EC2 CPU Monitoring

Screenshot 15 — DevNotes High CPU Alarm Configuration

What I did: Created a metric alarm named `DevNotes-High-CPU-Alarm` to monitor CPU utilization of the DevNotes EC2 instance.

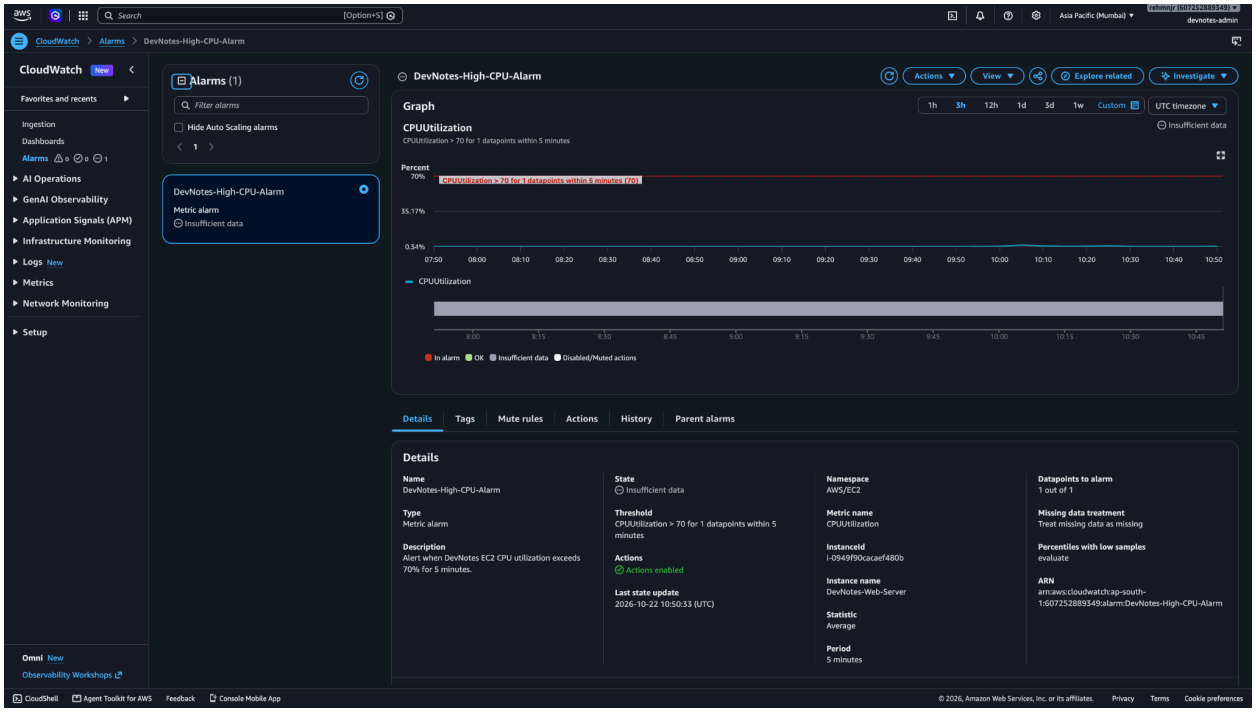
Why did it: To detect high CPU usage and enable notifications when the server's resource consumption exceeds the configured threshold.

Configuration :

- **Alarm name:** `DevNotes-High-CPU-Alarm`
- **EC2 instance:** `DevNotes-Web-Server`
- **Instance ID:** `i-0949f90cacaef480b`
- **Namespace:** `AWS/EC2`
- **Metric:** `CPUUtilization`
- **Statistic:** Average
- **Threshold:** CPU utilization greater than 70%
- **Period:** 5 minutes
- **Datapoints to alarm:** 1 out of 1
- **Missing data treatment:** Treat missing data as missing
- **Alarm actions:** Enabled
- **State shown:** Insufficient data

Result: The alarm was configured with a 70% CPU threshold and a five-minute evaluation period. At the time of this screenshot, CloudWatch displayed `Insufficient data`, so the alarm's current evaluation could not be confirmed.

Note: Alarm actions were enabled. The SNS topic and confirmed email subscription were configured separately to support email notifications.



IAM — EC2 Role and Custom Policies

What I did: Created the IAM role `DevNotes-EC2-S3-Role` and attached two custom inline policies.

Why did it: To allow the EC2 application server to access required AWS resources through an IAM role instead of storing long-term AWS access keys on the server.

Configuration:

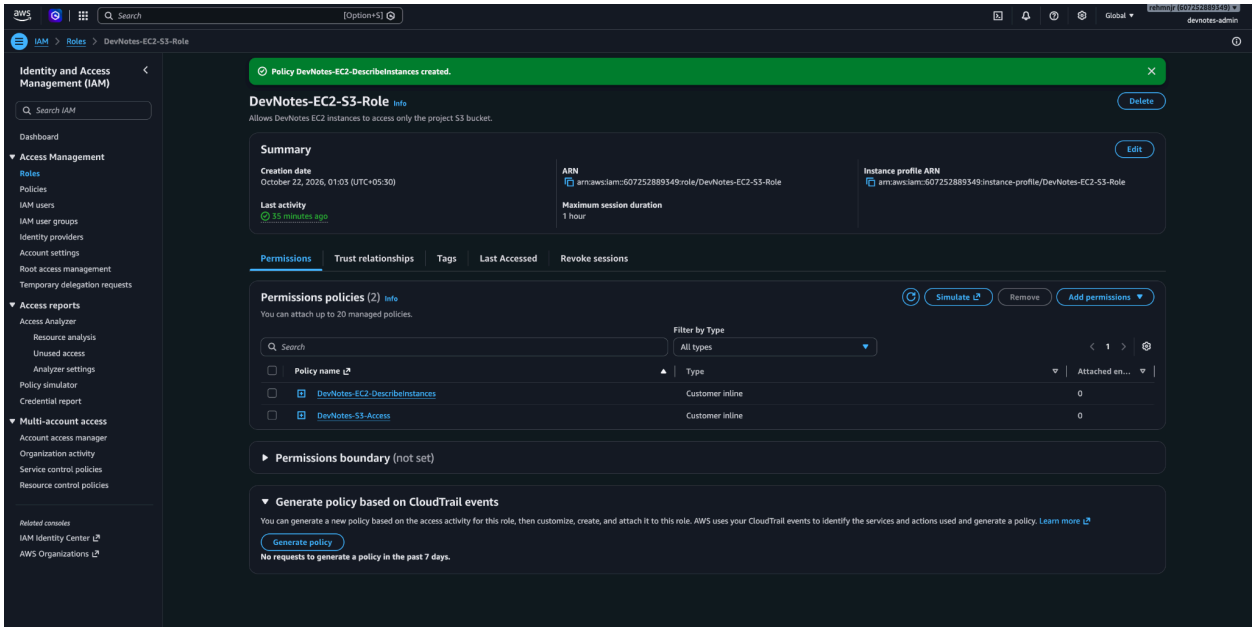
- **Role name:** `DevNotes-EC2-S3-Role`
- **Trusted service:** EC2 (as established in the role configuration)
- **Instance profile:** `DevNotes-EC2-S3-Role`
- **Maximum session duration:** 1 hour
- **Inline policy 1:** `DevNotes-S3-Access`
- **Inline policy 2:** `DevNotes-EC2-DescribeInstances`
- **Permissions boundary:** Not set

Purpose of the policies:

- `DevNotes-S3-Access` — grants the S3 permissions defined in the policy for the project's backup bucket.
- `DevNotes-EC2-DescribeInstances` — grants permission to describe EC2 instances.

Result: The IAM role and both inline policies were present in the AWS console.

Security note: The exact actions and resource scope must be verified by opening each policy. The role screenshot alone does not show the full policy JSON or prove that every permission is restricted to the intended resources.



Amazon S3 — Database Backup Verification

What I did: Stored two database backup files in the `backups/` folder of the DevNotes S3 bucket.

Why did it: To keep backup copies separate from the EC2 instance, so they can be recovered even if the application server is terminated.

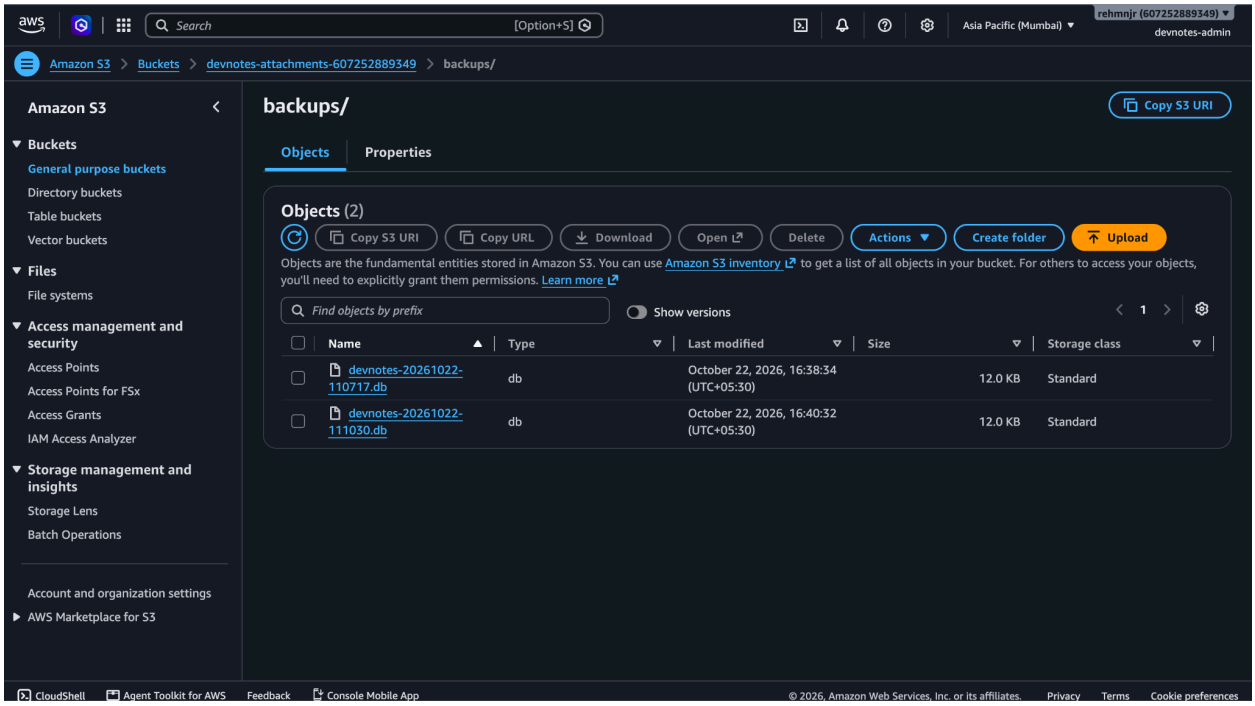
Configuration:

- **Bucket:** `devnotes-attachments-607252889349`
- **Prefix:** `backups/`
- **Backup files:** `devnotes-20261022-110717.db` and `devnotes-20261022-111030.db`

- **Size:** 12 KB each
- **Storage class:** S3 Standard

Result: Two backup files were present in S3. One file was downloaded to the local machine, and its SQLite integrity check returned `ok`.

Important note: These `.db` files are SQLite backups. Since DevNotes was migrated to MySQL on Amazon RDS, the SQLite backup should not be treated as a verified backup of the latest RDS database.




```
rehmnjr@MacJr DevNotes % ls -lh ~/Downloads/devnotes-20261022-110717.db
-rw-r--r--@ 1 rehmnjr  staff    12K Oct 23 23:56 /Users/rehmnjr/Downloads/devnotes-20261022-110717.db
rehmnjr@MacJr DevNotes % sqlite3 ~/Downloads/devnotes-20261022-110717.db "PRAGMA integrity_check;"
ok
rehmnjr@MacJr DevNotes %
```

```
rehmnjr@MacJr DevNotes % ls -lh ~/Downloads/devnotes-20261022-110717.db
-rw-r--r--@ 1 rehmnjr  staff    12K Oct 23 23:56 /Users/rehmnjr/Downloads/devnotes-20261022-110717.db
rehmnjr@MacJr DevNotes %
```

Amazon RDS — MySQL Database Setup

Configuration

- **AWS Service:** Amazon RDS
- **DB Identifier:** `database-1`
- **Database Engine:** MySQL-compatible engine
- **Status:** Available
- **Instance Role:** Instance
- **Availability Zone:** `ap-south-1b` (Mumbai Region)
- **Instance Class:** `db.t4g.micro` (as configured in the project)

What did I do?

We created a managed MySQL database using Amazon RDS for the DevNotes application. The database was configured to run in the Mumbai AWS Region.

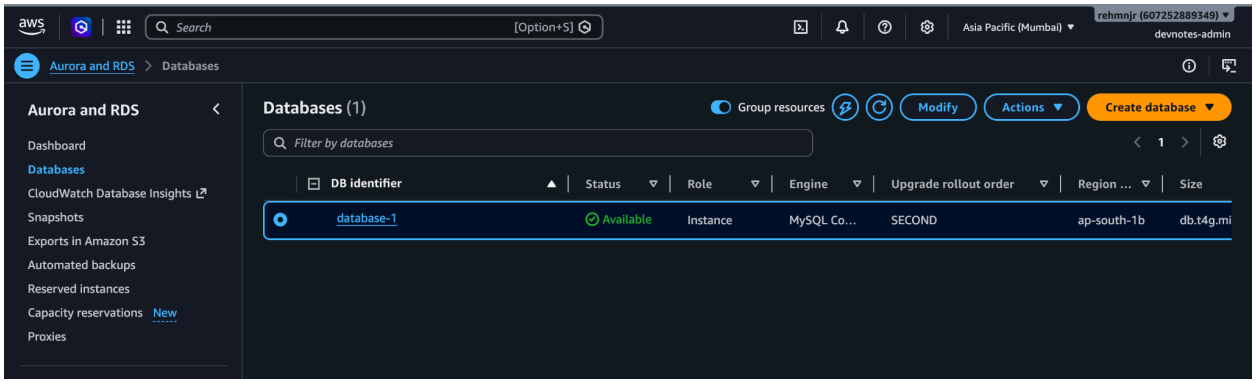
Why did I use RDS?

Amazon RDS manages database infrastructure and supports automated database maintenance and backups. It separates application compute from database storage, making the architecture easier to manage.

Result

The RDS database `database-1` is shown as **Available**, indicating that the database instance is ready for connections.

The DevNotes backend was configured to connect to this MySQL database instead of relying solely on its local SQLite database.



Amazon RDS — Connectivity & Security Configuration

1. Database Details

- **DB Identifier:** `database-1`
- **Engine:** MySQL Community
- **Instance Class:** `db.t4g.micro`
- **Status:** Available
- **AWS Region:** Asia Pacific (Mumbai)
- **Availability Zone:** `ap-south-1b`

2. Connection Configuration

- **Database Name:** `mysql` (as displayed in the connection panel)
- **Master Username:** `devnotesadmin`
- **Port:** `3306`
- **Internet Access Gateway:** Disabled
- **Endpoint:** `database-1.cluugk4m820m.ap-south-1.rds.amazonaws.com`

3. Networking Configuration

- **VPC:** `DevNotes-VPC-vpc`
- **VPC ID:** `vpc-0f132930898a5967e`
- **DB Subnet Group:** `default-vpc-0f132930898a5967e`
- **Network Type:** IPv4
- **Security Group:** `DevNotes-DB-SG`
- **Publicly Accessible:** No

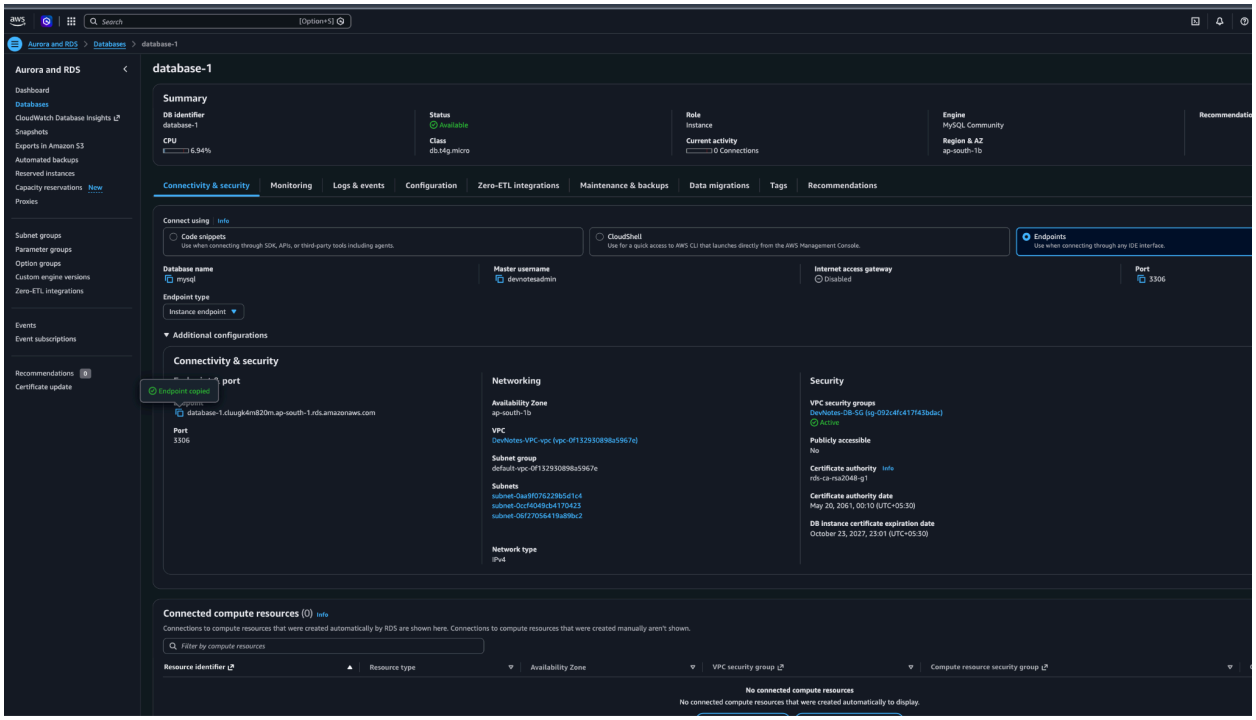
4. Security Explanation

The RDS database is configured as a private database and is not publicly accessible. The application server connects to it through the VPC network.

The database security group controls inbound database traffic on port `3306`. In our project, the intended configuration allows MySQL traffic from `DevNotes-Web-SG`, rather than opening the database port to everyone.

5. Project Outcome

The DevNotes backend was configured to use Amazon RDS MySQL for persistent data storage. This separates the database layer from the EC2 application server and provides a more structured cloud architecture.



GitHub Actions — CI Workflow Configuration

Repository: `rehmnjr/DevNotes`
Workflow file: `.github/workflows/deploy.yml`
Branch: `main`
Commit: `7d7b5be` — Add GitHub Actions CI workflow

What did I configure?

We created a GitHub Actions workflow in the `.github/workflows/` directory. The workflow file, `deploy.yml`, defines the automation used to validate the DevNotes project when the configured workflow triggers.

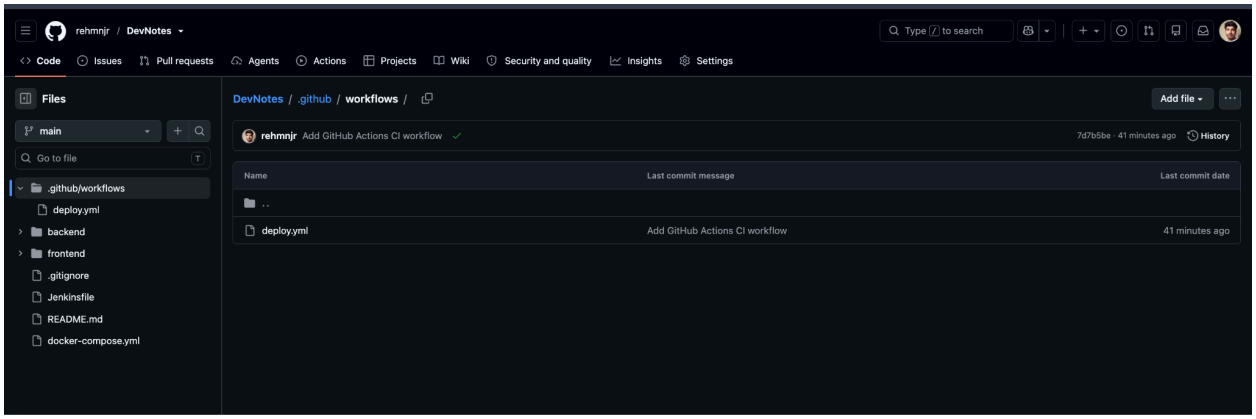
How does it work?

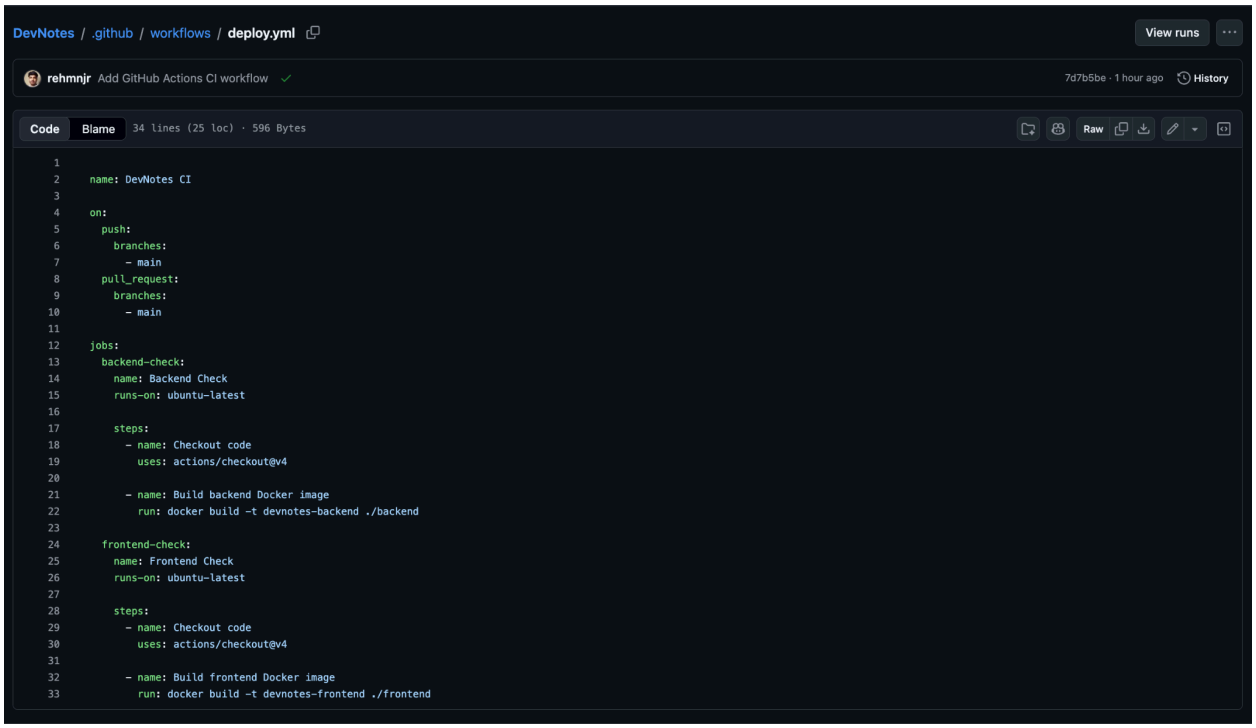
1. **Code Repository:** The DevNotes source code is maintained on GitHub.
2. **Workflow Configuration:** The `deploy.yml` file defines the CI workflow.

3. **Automated Checks:** The workflow runs the configured backend and frontend checks.
4. **Execution Status:** GitHub Actions displays whether the workflow jobs pass or fail.

Project Outcome

GitHub Actions was integrated into the DevNotes repository to automate backend and frontend validation. This helps identify build or configuration issues before changes are considered ready.





A. Workflow name

name: DevNotes CI

- `name`: Defines the workflow's display name.
- `DevNotes CI`: The name shown in the GitHub Actions tab.

- `CI` means Continuous Integration.

In simple words: This identifies the automation workflow for the DevNotes project.

B. Workflow triggers

```
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
```

Line	Explanation
`on:`	Defines when the workflow should run.
`push:`	Triggers the workflow when code is pushed.
`branches:`	Specifies which branches to monitor.
`- main`	Runs for pushes to the `main` branch.
`pull_request:`	Triggers the workflow for pull requests targeting `main`.
`- main`	Specifies `main` as the target branch for pull requests.

Example: You push a new commit to `main`. GitHub Actions starts the workflow automatically. If someone opens or updates a pull request targeting `main`, the workflow also runs.

C. Jobs section

```
jobs:
```

- `jobs` defines the tasks that GitHub Actions must execute.
- This workflow contains two jobs: `backend-check` and `frontend-check`.
- GitHub Actions can run these jobs in parallel because neither depends on the other.

D. Backend job

```
backend-check:
  name: Backend Check
  runs-on: ubuntu-latest
```

- `backend-check`: The unique ID of this job, used internally by GitHub Actions.
- `name: Backend Check`: The readable name displayed in the Actions interface.
- `runs-on: ubuntu-latest`: Runs the job on a GitHub-hosted virtual machine using the latest available Ubuntu runner image.

In simple words: GitHub provides a temporary Ubuntu machine to check whether the backend Docker image can be built.

E. Steps section

```
steps:
```

- `steps` contains the individual actions and commands that the job executes.
- Steps in a job run sequentially, from top to bottom.

F. Checkout code

```
- name: Checkout code
  uses: actions/checkout@v4
```

- `-`: Starts a new step in the list.
- `name: Checkout code`: Gives the step a descriptive name.
- `uses:`: Tells GitHub Actions to use an existing action.
- `actions/checkout@v4`: Uses version 4 of GitHub's official checkout action to download the repository's source code into the runner's workspace.

Why is this necessary? The runner needs your project files before it can build the Docker image.

G. Build backend Docker image

```
- name: Build backend Docker image
  run: docker build -t devnotes-backend ./backend
```

- `name: Build backend Docker image`: Describes the purpose of the step.
- `run:`: Executes a shell command on the runner.
- `docker build`: Builds a Docker image using a Dockerfile.

- `-t devnotes-backend`: Tags the resulting image with the name `devnotes-backend`.
- `./backend`: Specifies the `backend` directory as the Docker build context. Docker uses this directory's files and its Dockerfile, if present, to build the image.

What happens? GitHub Actions attempts to build your backend image. If the command fails, the step and normally the job are marked as failed.

H. Frontend job

frontend-check:
  name: Frontend Check
  runs-on: ubuntu-latest

This is the second job.

- `frontend-check`: The job's unique ID.
- `name: Frontend Check`: The display name.
- `runs-on: ubuntu-latest`: Uses an Ubuntu runner for the frontend check.

It is independent of the backend job, so both jobs can run at the same time.

I. Frontend checkout

steps:
  - name: Checkout code
    uses: actions/checkout@v4

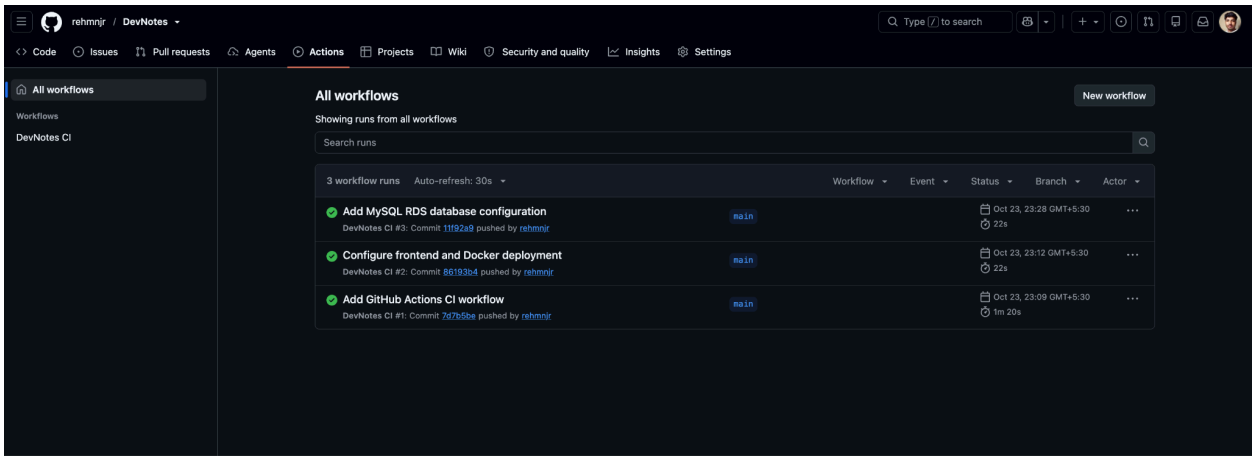
This performs the same checkout operation as the backend job, but in the frontend job's own runner workspace.

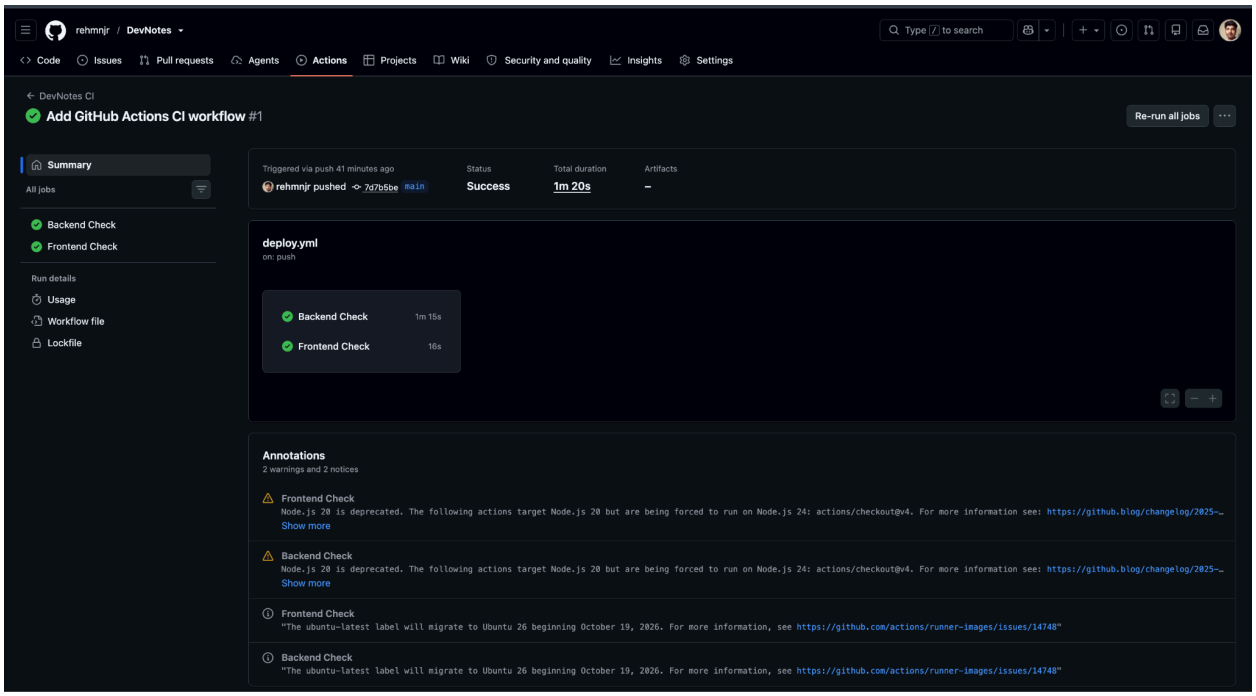
Important: Each job gets its own runner environment, so the frontend job also needs to check out the repository.

J. Build frontend Docker image

- name: Build frontend Docker image
  run: docker build -t devnotes-frontend ./frontend

- `name`: Describes the frontend build step.
- `run`: Executes the Docker command.
- `docker build`: Builds the frontend Docker image.
- `-t devnotes-frontend`: Names the resulting image `devnotes-frontend`.
- `./frontend`: Uses the `frontend` directory as the Docker build context.





Jenkins CI/CD Pipeline — DevNotes

1. Jenkins Pipeline Overview

Pipeline Name: `DevNotes-CI`

Description: CI/CD pipeline for DevNotes — React frontend, FastAPI backend, and Docker Compose.

Jenkins is used to automate the application's build workflow. The pipeline retrieves source code from GitHub, verifies the project configuration, builds Docker images, and pushes images to Docker Hub.

Pipeline Execution Stages

Stage	Purpose
Declarative: Checkout SCM	Initializes the pipeline and checks out source code from the configured Source Code Management (SCM) system.
Checkout	Retrieves the repository code from GitHub.
Verify Project Structure	Checks whether the expected project files and directories exist.
Verify Docker Compose Config	Validates the Docker Compose configuration.
Build Backend Image	Builds the backend Docker image.
Build Frontend Image	Builds the frontend Docker image.
Push Images to Docker Hub	Uploads the built images to Docker Hub.
Declarative: Post Actions	Executes post-build actions defined in the Jenkinsfile.

Build Result

The Stage View shows build **#6** with all displayed stages in green, indicating that the pipeline completed successfully.

The screenshot also shows earlier successful and failed builds, demonstrating that Jenkins maintains build history for troubleshooting and tracking changes.

2. Jenkins Job Configuration

General

- **Job Name:** `DevNotes-CI`
- **Status:** Enabled

- **Description:** CI/CD pipeline for DevNotes — React frontend, FastAPI backend, and Docker Compose.

Source Code Management (SCM)

- **SCM Type:** Git
- **Repository URL:** `https://github.com/rehmnjr/DevNotes.git`
- **Branch Specifier:** `*/main`
- **Script Path:** `Jenkinsfile`
- **Lightweight Checkout:** Enabled

Explanation: Jenkins retrieves the project from the GitHub repository and reads the pipeline instructions from the `Jenkinsfile` located at the repository root.

Build Triggers

The visible trigger options are unchecked in the screenshot. Therefore, this configuration does not show a scheduled build or a configured polling/webhook trigger.

The pipeline can still be started manually using **Build Now**.

General Job Options

The visible checkboxes for discarding old builds, preventing concurrent builds, and other optional job settings are unchecked.

Pipeline Definition

- **Definition:** Pipeline script from SCM
- **SCM:** Git
- **Repository:** DevNotes GitHub repository
- **Branch:** `*/main`
- **Script Path:** `Jenkinsfile`

This configuration keeps the pipeline code inside the GitHub repository rather than requiring the entire pipeline script to be entered directly in the Jenkins UI.

3. Overall Workflow

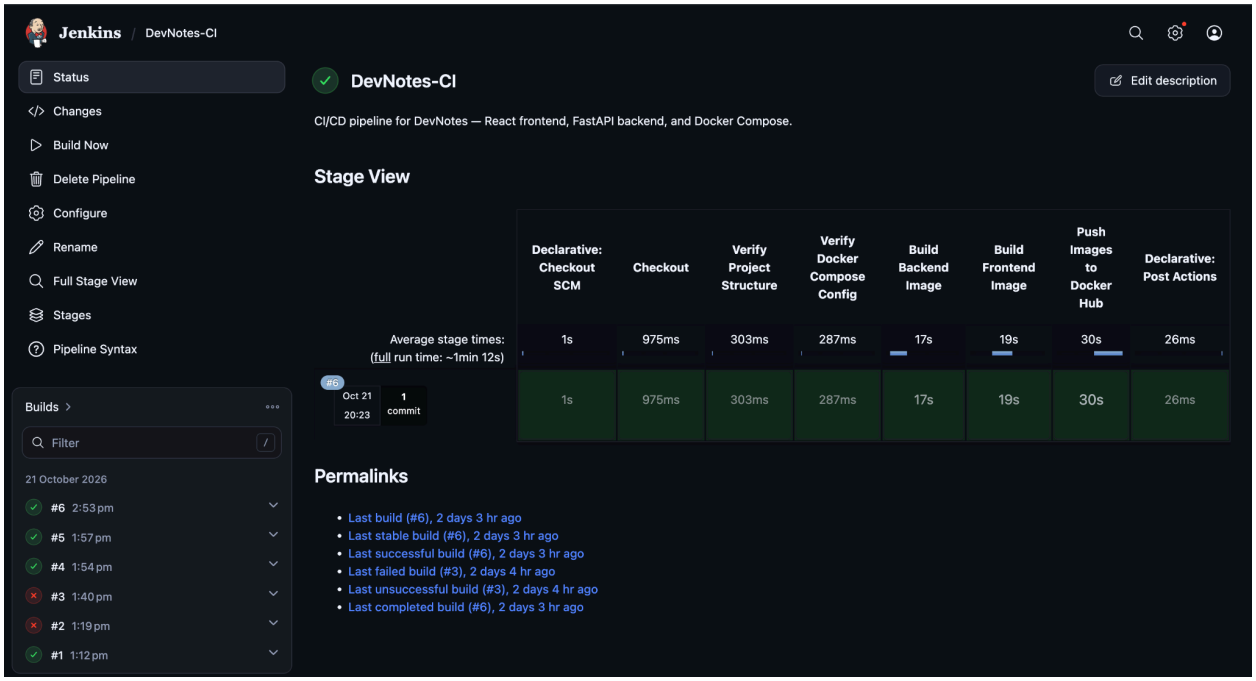
1. Jenkins checks out the DevNotes repository.
2. It verifies the project structure and Docker Compose configuration.
3. It builds the backend and frontend Docker images.
4. It pushes the images to Docker Hub.

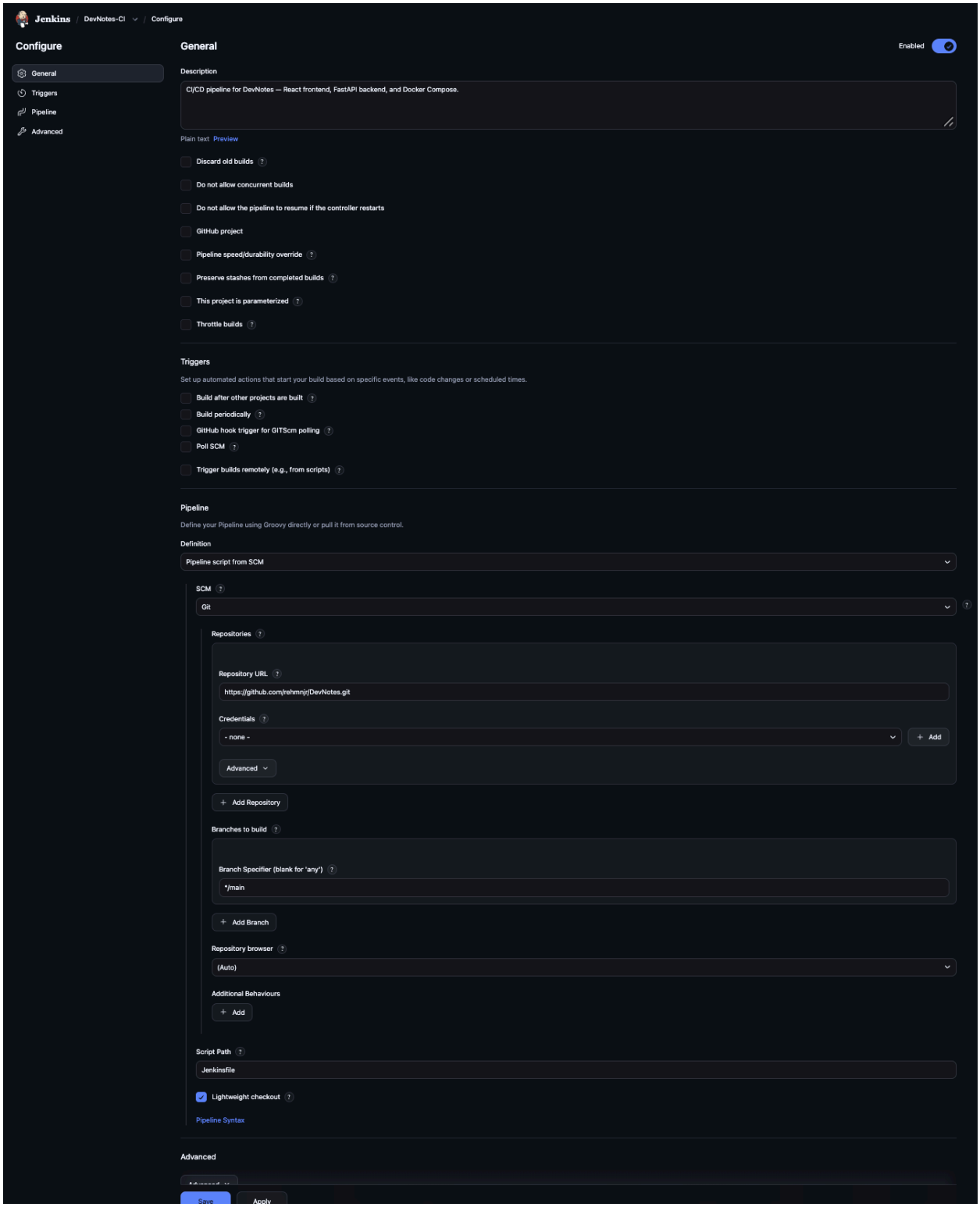
5. It executes the configured post-build actions.
6. Jenkins records the result and displays each stage in Stage View.

4. Conclusion

Jenkins is integrated with the DevNotes GitHub repository through a `Jenkinsfile` stored in source control. The pipeline automates Docker image building and publishing, while Stage View provides visibility into each stage and its execution status.

Evidence: The first screenshot documents the successful pipeline execution. The second documents the Jenkins job configuration and Git repository integration.




Jenkins / DevNotes-CI / Configure
Configure
General
Triggers
Pipeline
Advanced
General
Enabled
Description
CI/CD pipeline for DevNotes — React frontend, FastAPI backend, and Docker Compose.
Plain text Preview
Discard old builds
Do not allow concurrent builds
Do not allow the pipeline to resume if the controller restarts
GitHub project
Pipeline speed/durability override
Preserve stashes from completed builds
This project is parameterized
Throttle builds
Triggers
Set up automated actions that start your build based on specific events, like code changes or scheduled times.
Build after other projects are built
Build periodically
GitHub hook trigger for GITScm polling
Poll SCM
Trigger builds remotely (e.g., from scripts)
Pipeline
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script from SCM
SCM
Git
Repositories
Repository URL
https://github.com/rehmnjr/DevNotes.git
Credentials
- none -
Add
Advanced
Add Repository
Branches to build
Branch Specifier (blank for 'any')
*/main
Add Branch
Repository browser
(Auto)
Additional Behaviours
Add
Script Path
Jenkinsfile
Lightweight checkout
Pipeline Syntax
Advanced
Save
Apply

DOCKER

Step 1: Docker Hub Repositories

Docker Hub contains two repositories: `devnotes-backend` and `devnotes-frontend`. These repositories store the Docker images of the backend and frontend services, which can be pulled and deployed on the EC2 instance.

Step 2: Checking Running Docker Containers

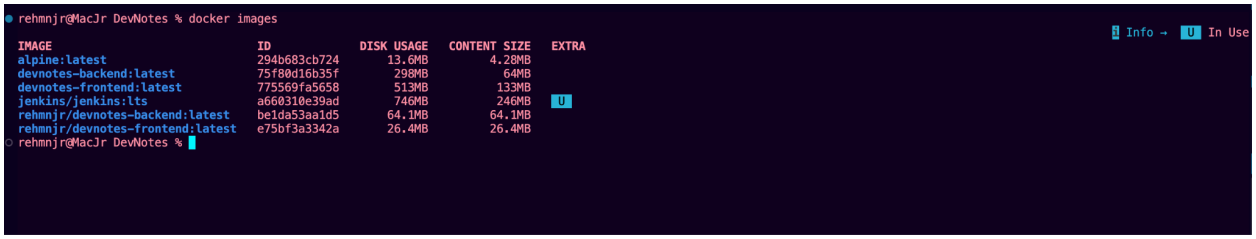
The `docker ps -a` command displays all Docker containers, including running and stopped containers. The output shows the Jenkins server, DevNotes UI, and DevNotes API containers running successfully on the local machine, along with their exposed ports.

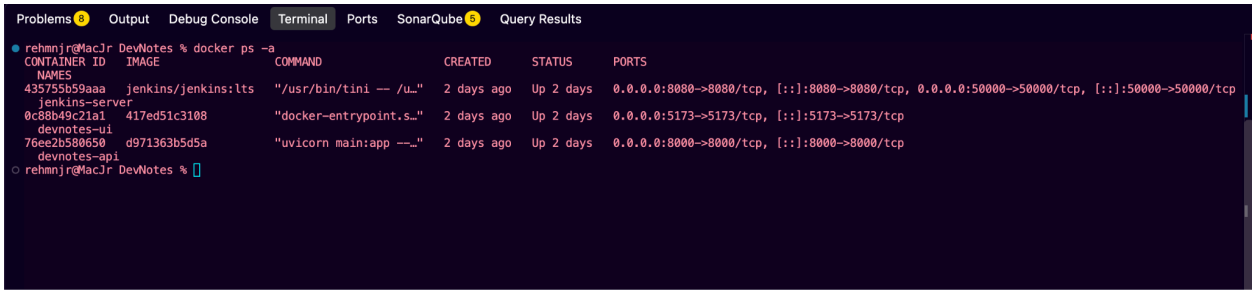
Step 3: Listing Docker Images

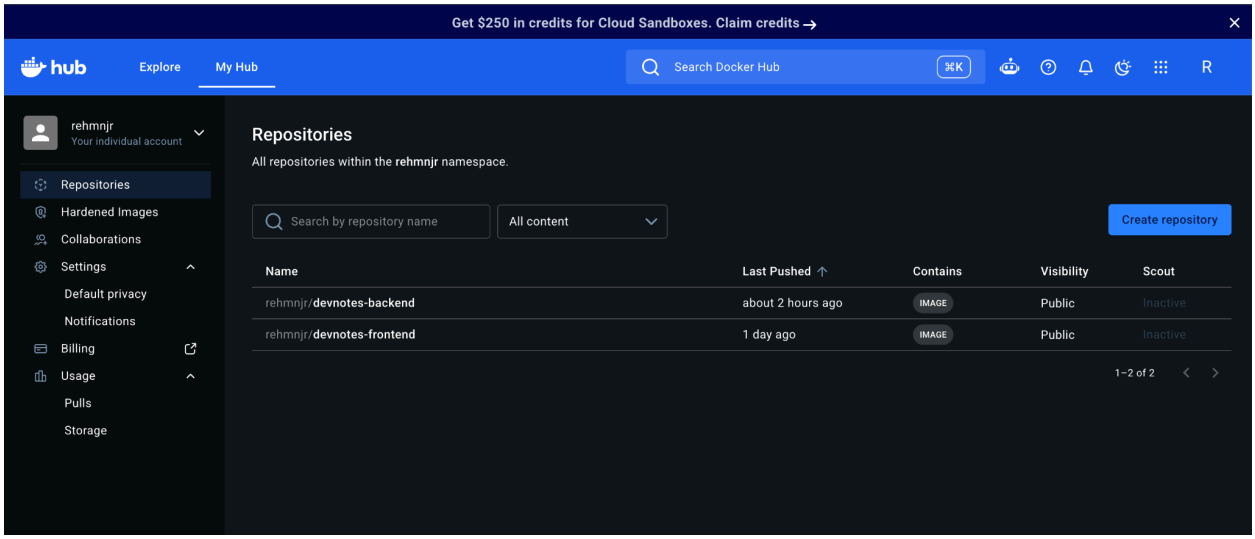
The `docker images` command lists the Docker images available locally. The output includes the DevNotes backend and frontend images, their Docker Hub-tagged versions, the Jenkins image, and the Alpine image.

Step 4: Checking Docker Compose on EC2

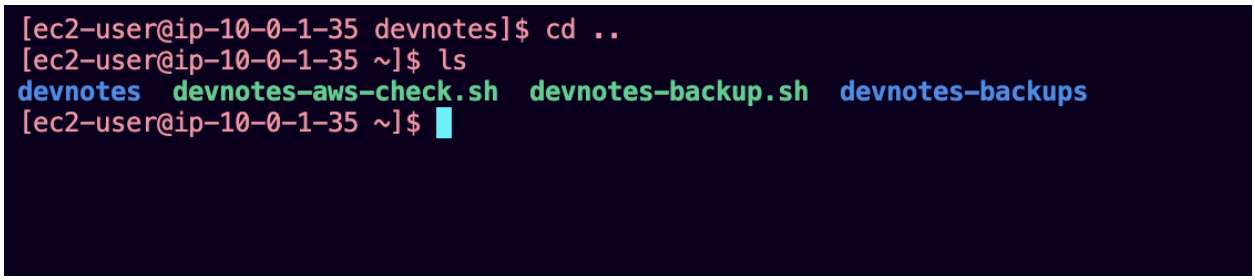
The `docker compose ps` command returned an error because the Docker CLI could not recognize the Compose command. This indicates that the Compose v2 plugin may not be installed or available. The existing application status should be verified before making any configuration changes.




Problems 8
Output
Debug Console
Terminal
Ports
SonarQube 5
Query Results
rehmnjr@MacJr DevNotes % docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
435755b59aaa jenkins/jenkins:lts "/usr/bin/tini -- /u…" 2 days ago Up 2 days 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp jenkins-server
0c88b49c21a1 417ed51c3108 "docker-entrypoint.s…" 2 days ago Up 2 days 0.0.0.0:5173->5173/tcp, [::]:5173->5173/tcp devnotes-ui
76ee2b580650 d971363b5d5a "uvicorn main:app --…" 2 days ago Up 2 days 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp devnotes-api
rehmnjr@MacJr DevNotes %


Get $250 in credits for Cloud Sandboxes. Claim credits →
hub
Explore
My Hub
Search Docker Hub
⌘K
R
rehmnjr
Your individual account
Repositories
Hardened Images
Collaborations
Settings
Default privacy
Notifications
Billing
Usage
Pulls
Storage
Repositories
All repositories within the rehmnjr namespace.
Search by repository name
All content
Create repository
Name
Last Pushed
Contains
Visibility
Scout
rehmnjr/devnotes-backend
about 2 hours ago
IMAGE
Public
Inactive
rehmnjr/devnotes-frontend
1 day ago
IMAGE
Public
Inactive
1–2 of 2

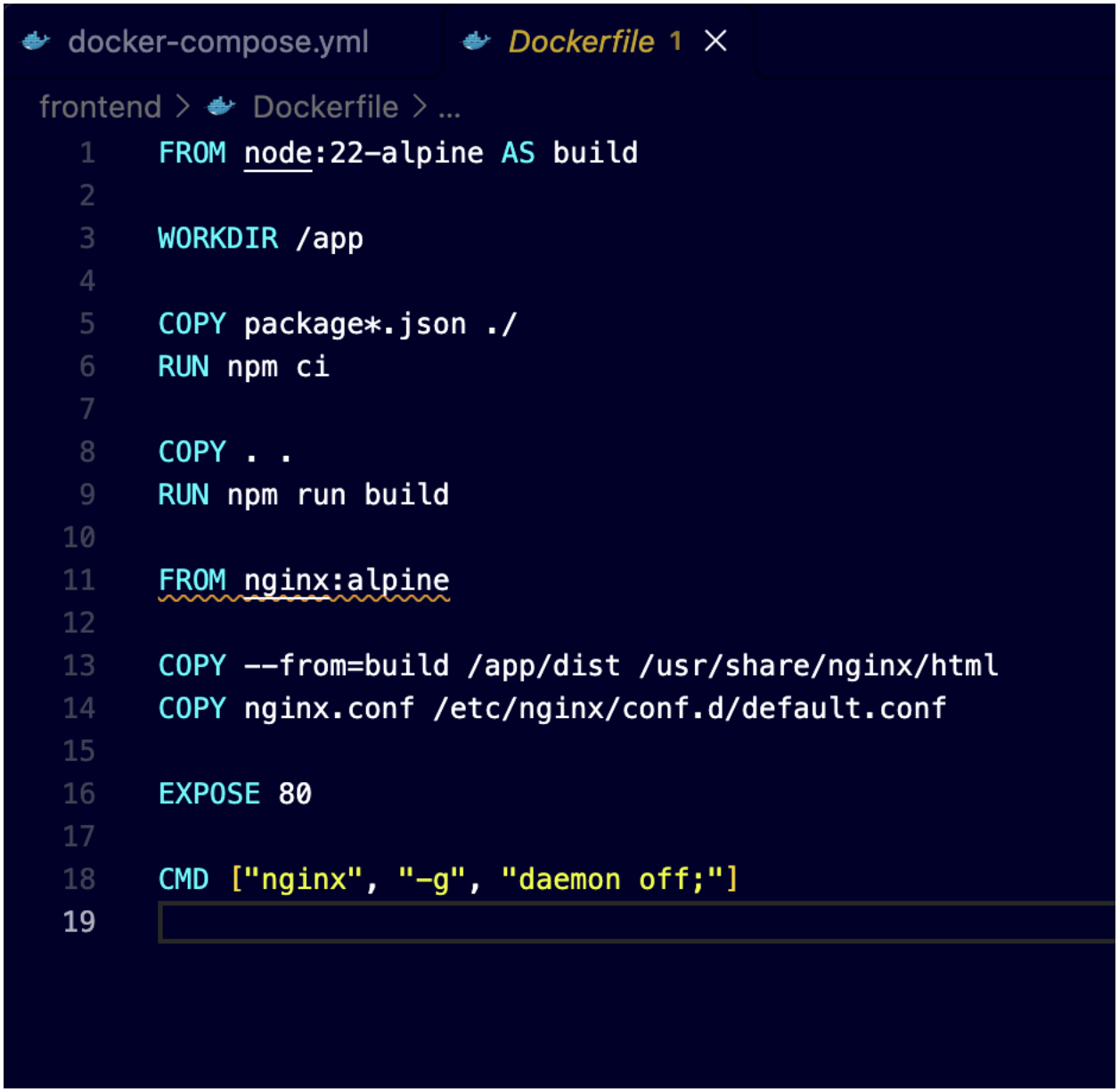

[ec2-user@ip-10-0-1-35 devnotes]$ docker-compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
devnotes-api rehmnjr/devnotes-backend:latest "uvicorn main:app --…" backend 2 hours ago Up 2 hours 8000/tcp
devnotes-ui rehmnjr/devnotes-frontend:latest "/docker-entrypoint.…" frontend 33 hours ago Up 33 hours 0.0.0.0:80->80/tcp, [::]:80->80/tcp
[ec2-user@ip-10-0-1-35 devnotes]$


[ec2-user@ip-10-0-1-35 devnotes]$ cd ..
[ec2-user@ip-10-0-1-35 ~]$ ls
devnotes devnotes-aws-check.sh devnotes-backup.sh devnotes-backups
[ec2-user@ip-10-0-1-35 ~]$

```
[ec2-user@ip-10-0-1-35 devnotes]$ ls
docker-compose.yml
[ec2-user@ip-10-0-1-35 devnotes]$ cat docker-compose.yml
services:
  backend:
    image: rehmnjr/devnotes-backend:latest
    container_name: devnotes-api
    env_file:
      - .env
    volumes:
      - devnotes_data:/data
    expose:
      - "8000"
    restart: unless-stopped

  frontend:
    image: rehmnjr/devnotes-frontend:latest
    container_name: devnotes-ui
    ports:
      - "80:80"
    depends_on:
      - backend
    restart: unless-stopped

volumes:
  devnotes_data:
[ec2-user@ip-10-0-1-35 devnotes]$
```


docker-compose.yml
Dockerfile 1
frontend > Dockerfile > ...
1 FROM node:22-alpine AS build
2
3 WORKDIR /app
4
5 COPY package*.json ./
6 RUN npm ci
7
8 COPY . .
9 RUN npm run build
10
11 FROM nginx:alpine
12
13 COPY --from=build /app/dist /usr/share/nginx/html
14 COPY nginx.conf /etc/nginx/conf.d/default.conf
15
16 EXPOSE 80
17
18 CMD ["nginx", "-g", "daemon off;"]
19

docker-compose.yml | Dockerfile 1

backend > Dockerfile > ...

```
FROM python:3.12-slim

WORKDIR /app

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

COPY requirements.txt .

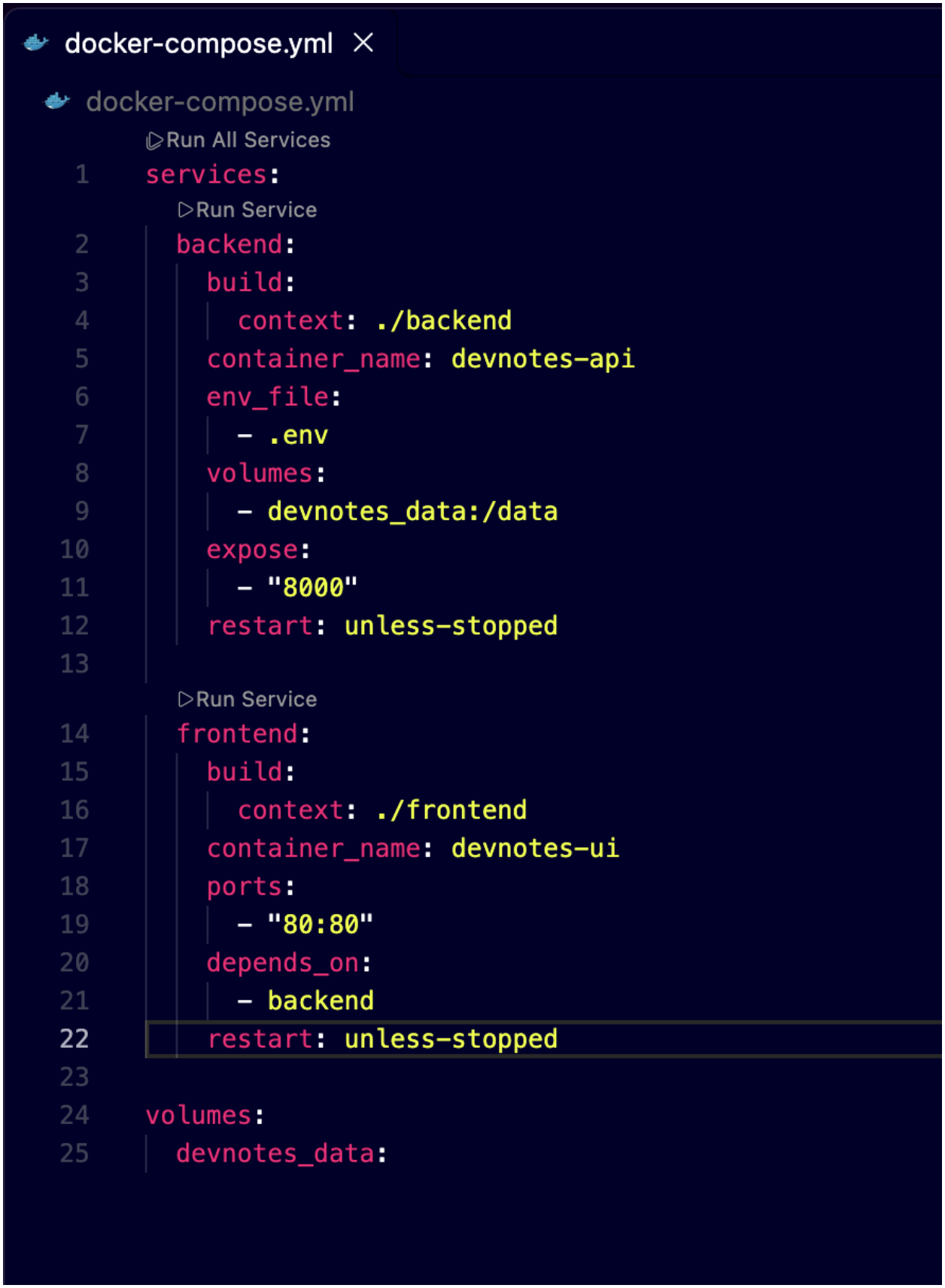
RUN pip install --no-cache-dir -r requirements.txt

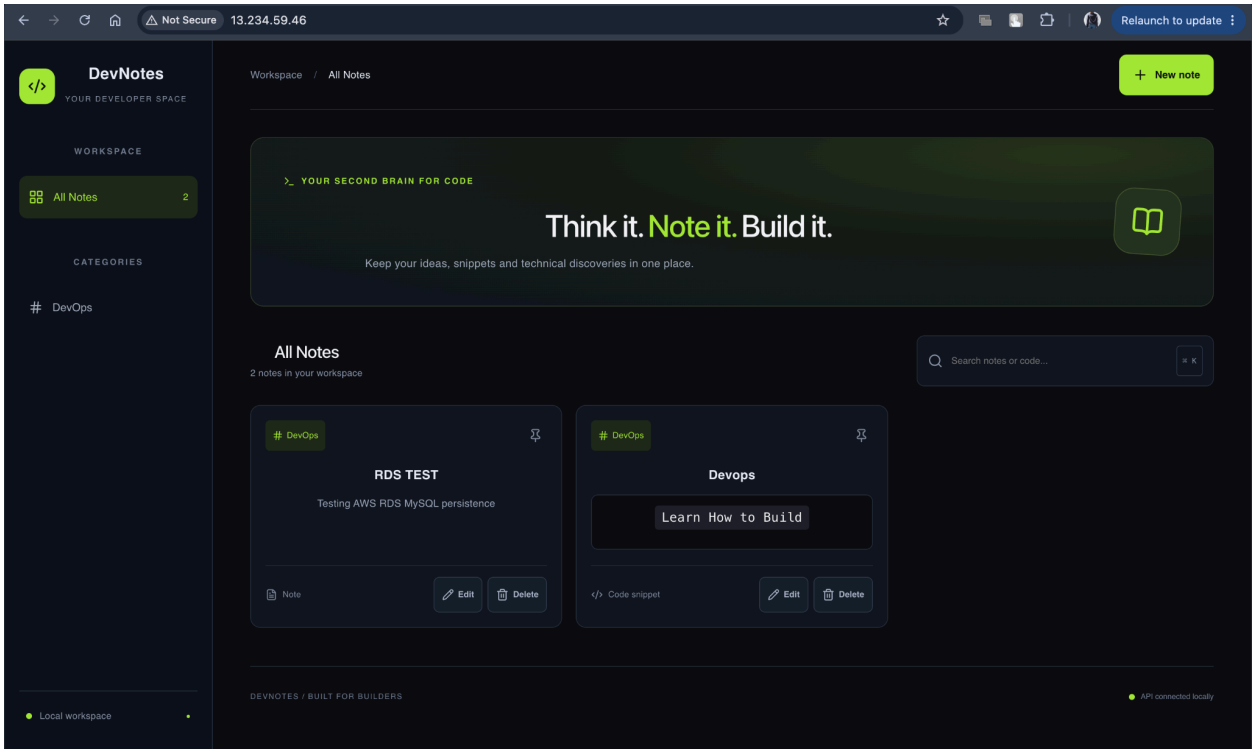
COPY main.py .

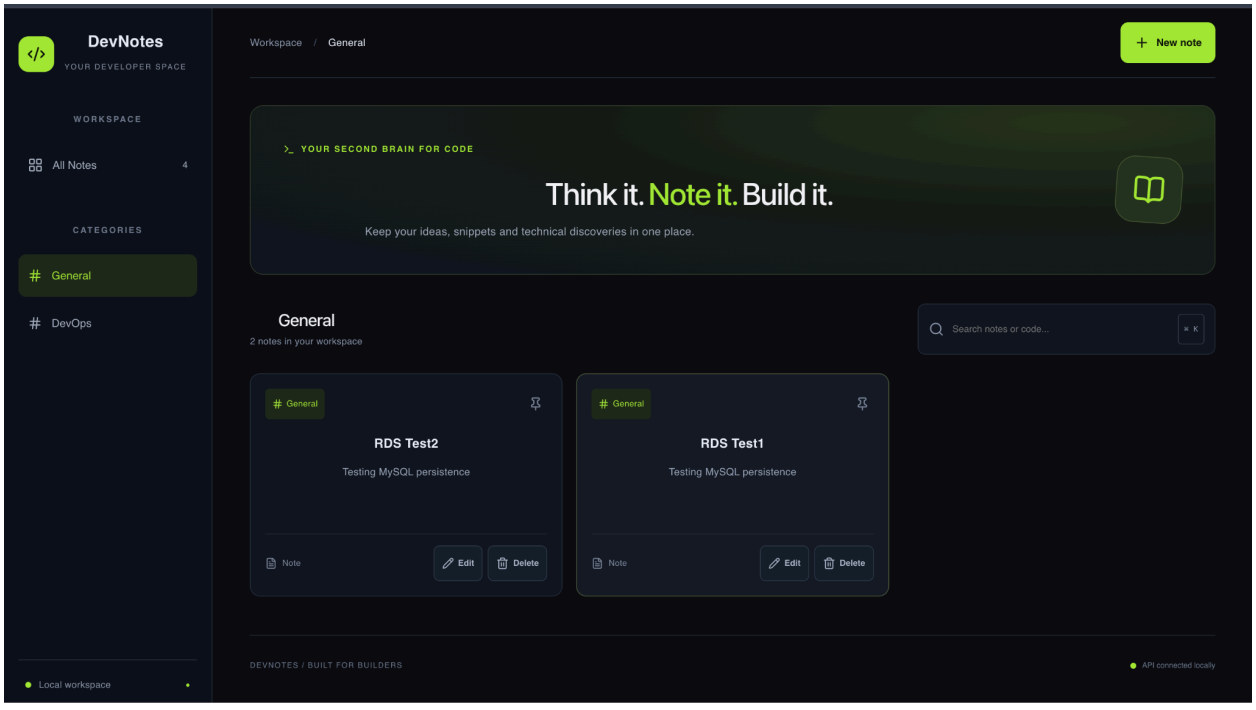
EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```


docker-compose.yml
docker-compose.yml
Run All Services
services:
Run Service
backend:
build:
context: ./backend
container_name: devnotes-api
env_file:
- .env
volumes:
- devnotes_data:/data
expose:
- "8000"
restart: unless-stopped
Run Service
frontend:
build:
context: ./frontend
container_name: devnotes-ui
ports:
- "80:80"
depends_on:
- backend
restart: unless-stopped
volumes:
devnotes_data:


Not Secure
13.234.59.46
Relaunch to update
DevNotes
YOUR DEVELOPER SPACE
WORKSPACE
All Notes 2
CATEGORIES
DevOps
Local workspace
Workspace / All Notes
New note
YOUR SECOND BRAIN FOR CODE
Think it. Note it. Build it.
Keep your ideas, snippets and technical discoveries in one place.
All Notes
2 notes in your workspace
Search notes or code...
DevOps
RDS TEST
Testing AWS RDS MySQL persistence
Note
Edit
Delete
DevOps
Devops
Learn How to Build
Code snippet
Edit
Delete
DEVNOTES / BUILT FOR BUILDERS
API connected locally


DevNotes
YOUR DEVELOPER SPACE
WORKSPACE
All Notes 4
CATEGORIES
General
DevOps
Local workspace
Workspace / General
New note
YOUR SECOND BRAIN FOR CODE
Think it. Note it. Build it.
Keep your ideas, snippets and technical discoveries in one place.
General
2 notes in your workspace
Search notes or code...
General
RDS Test2
Testing MySQL persistence
Note
Edit
Delete
General
RDS Test1
Testing MySQL persistence
Note
Edit
Delete
DEVNOTES / BUILT FOR BUILDERS
API connected locally